Accelerating Memory Operations

The Memory Hierarchy

Prof. Chester Rebeiro

Indian Institute of Technology Madras

Mantras for speeding up CPUs

”Exploit the program’s Spatial Locality”

”Exploit the program’s Temporal Locality”

”Parallelize to speed up execution”

”Leave no hardware module unused”

Complications realizing the mantras

Mantras

”Exploit the program’s Spatial Locality”

”Exploit the program’s Temporal Locality”

”Parallelize to speed up execution”

”Leave no hardware module unused”

Complications

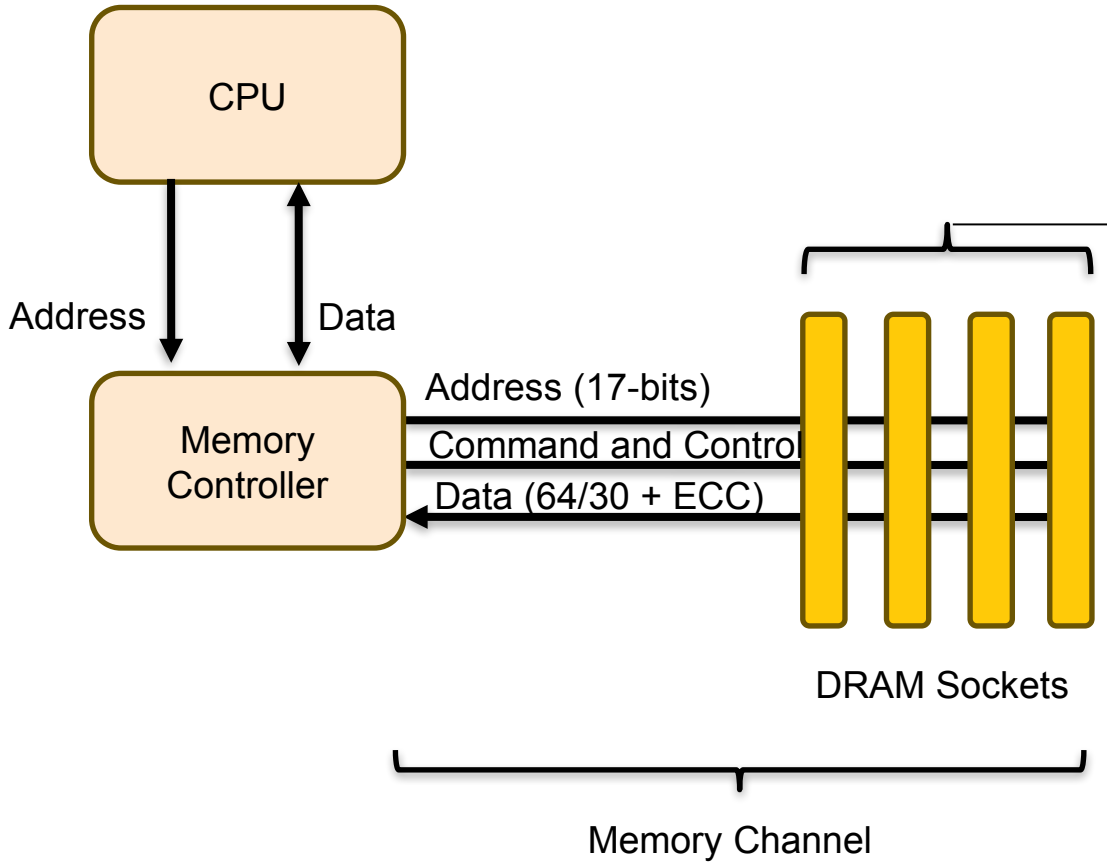
”Physical characteristics”

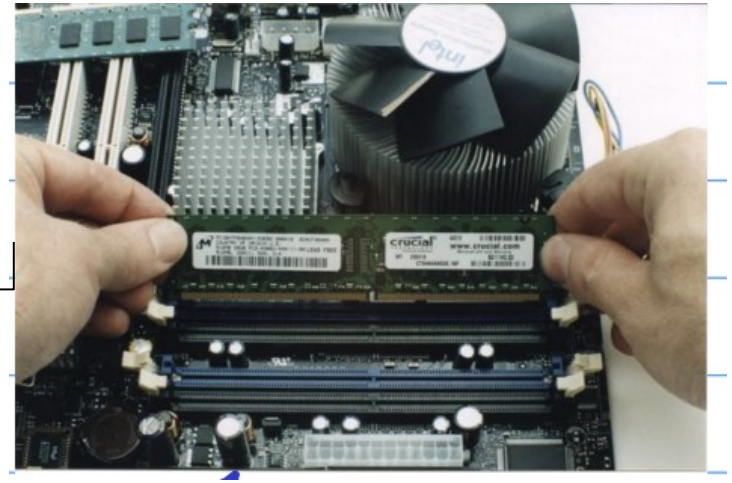
”Cost and size”

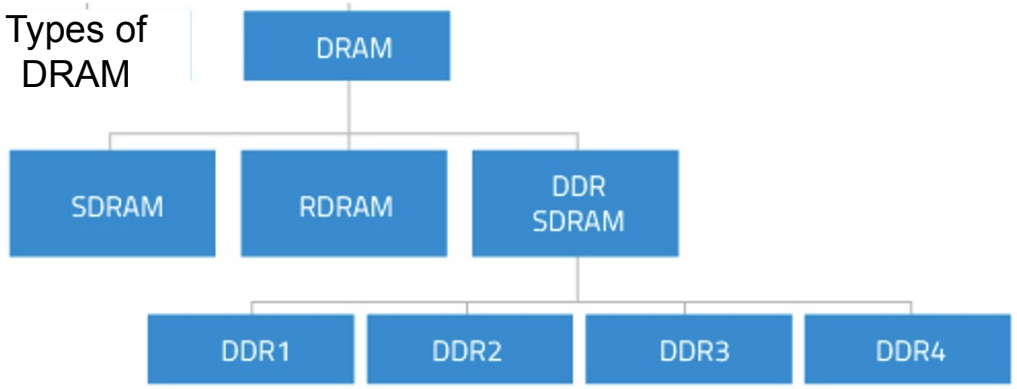
”Complexity of design”

DRAM Organization

Let's have a closer look at Memory



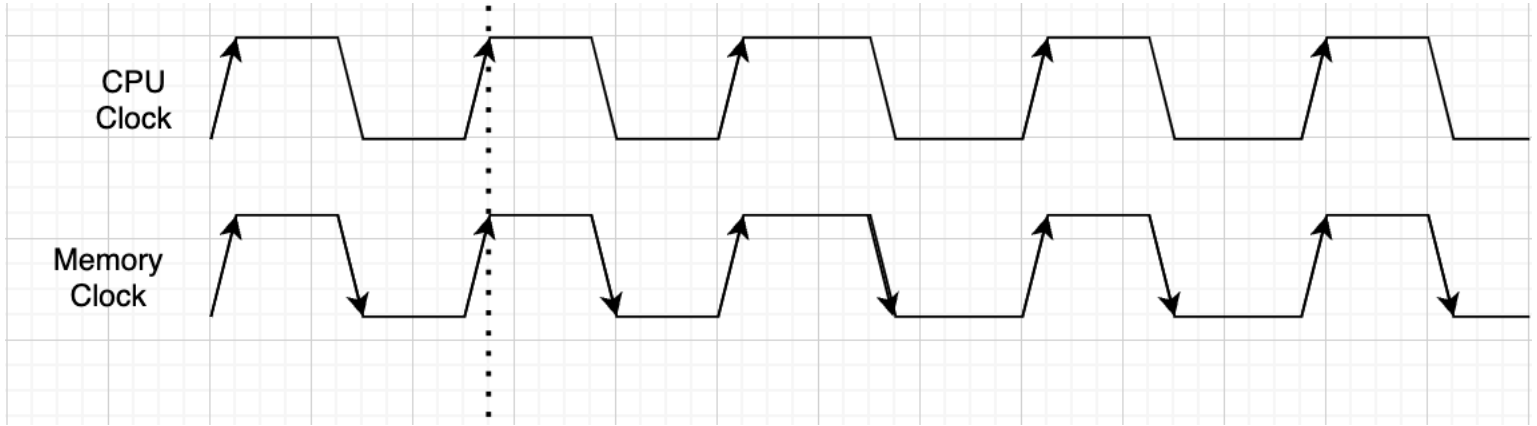




SDRAM: Synchronous DRAM
RDRAM: Rambus DRAM
DDRAM: Double Data Rate SDRAM

ECC: Error detection and correction

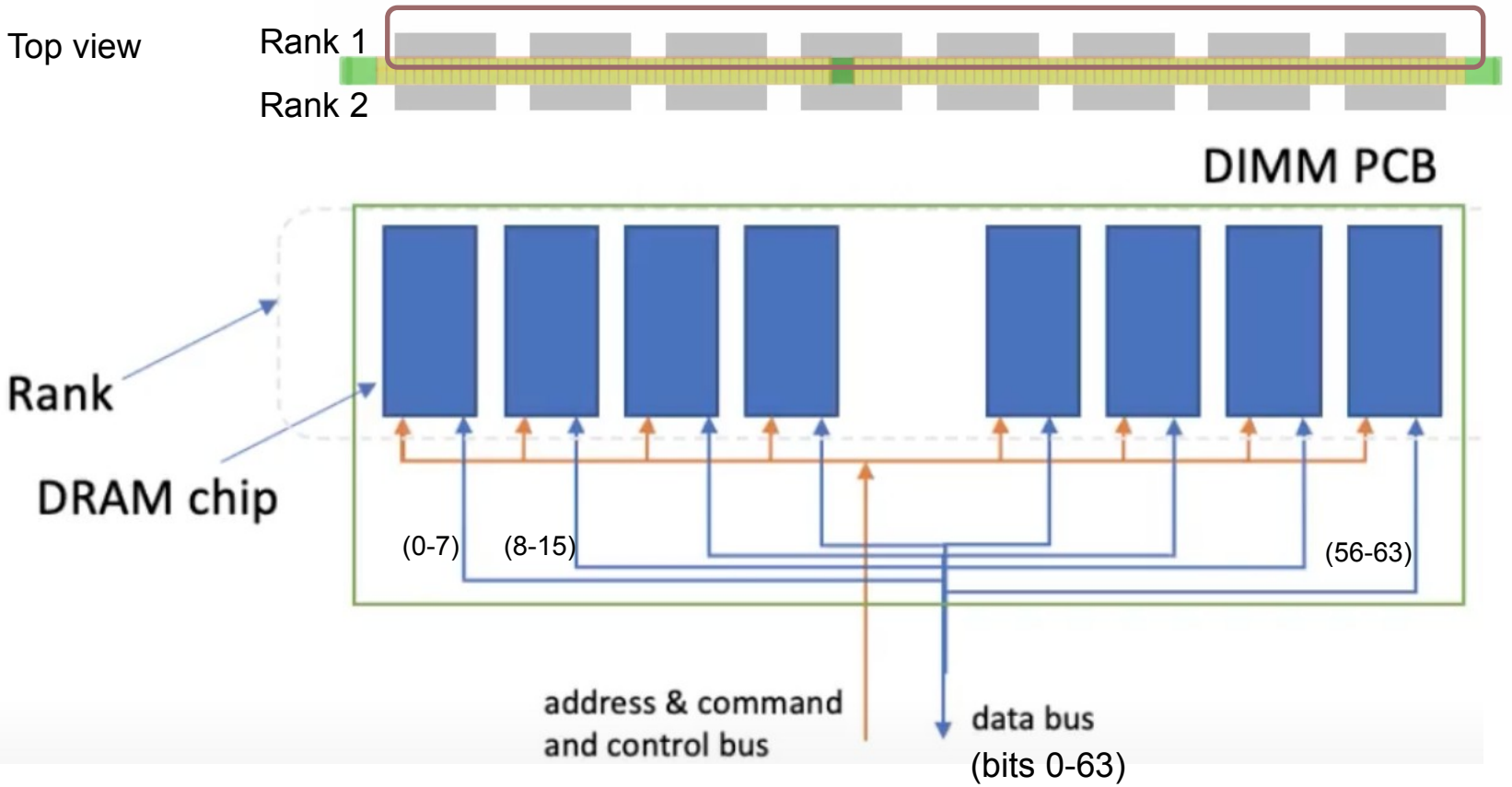
Synchronous and Double Data Rate

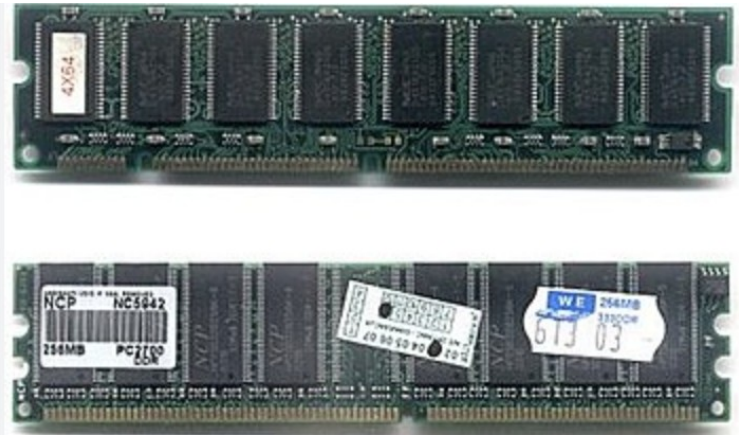


CPU clock synchronized with Memory clock

Double Data Rate: Data read at both rising edge and falling edge of clock

DRAM Organization
Ranks and DRAM Chips

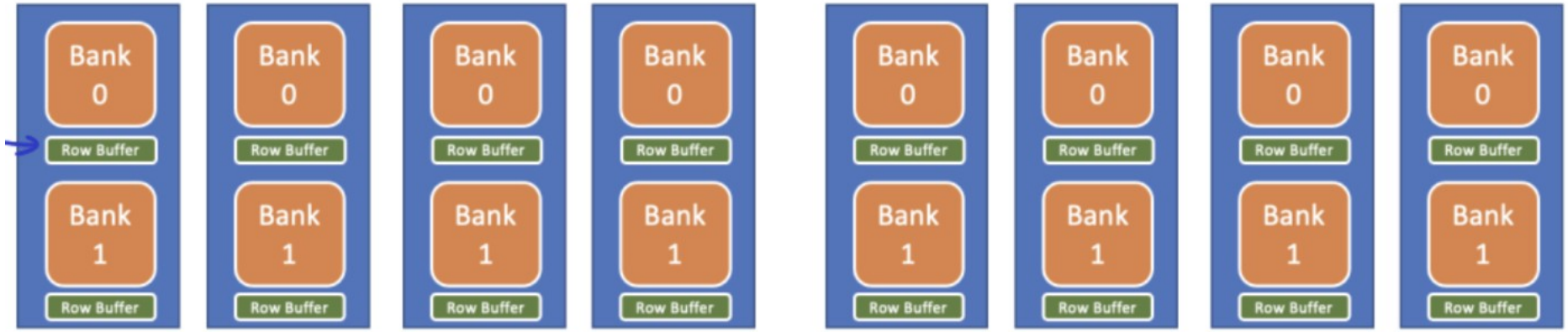




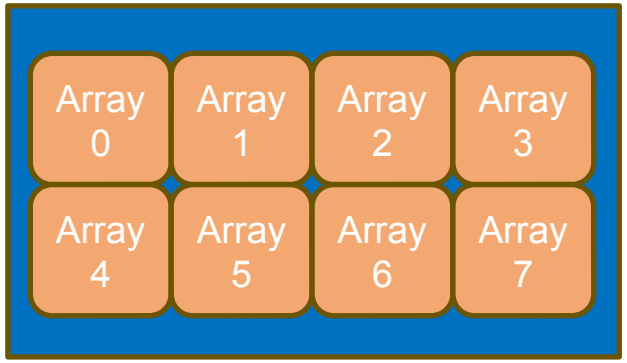
In this example, Rank has 8-chips, each supplying 8 bits.
Making a total of 64-bits on the data bus

Multiple chips work together to service a data request.
Pros. Lesser cost, Each DRAM chip just needs 8 pins for data
Cons. Each request activates all DRAM chips, hence more power

DRAM Organization
Banks and Arrays

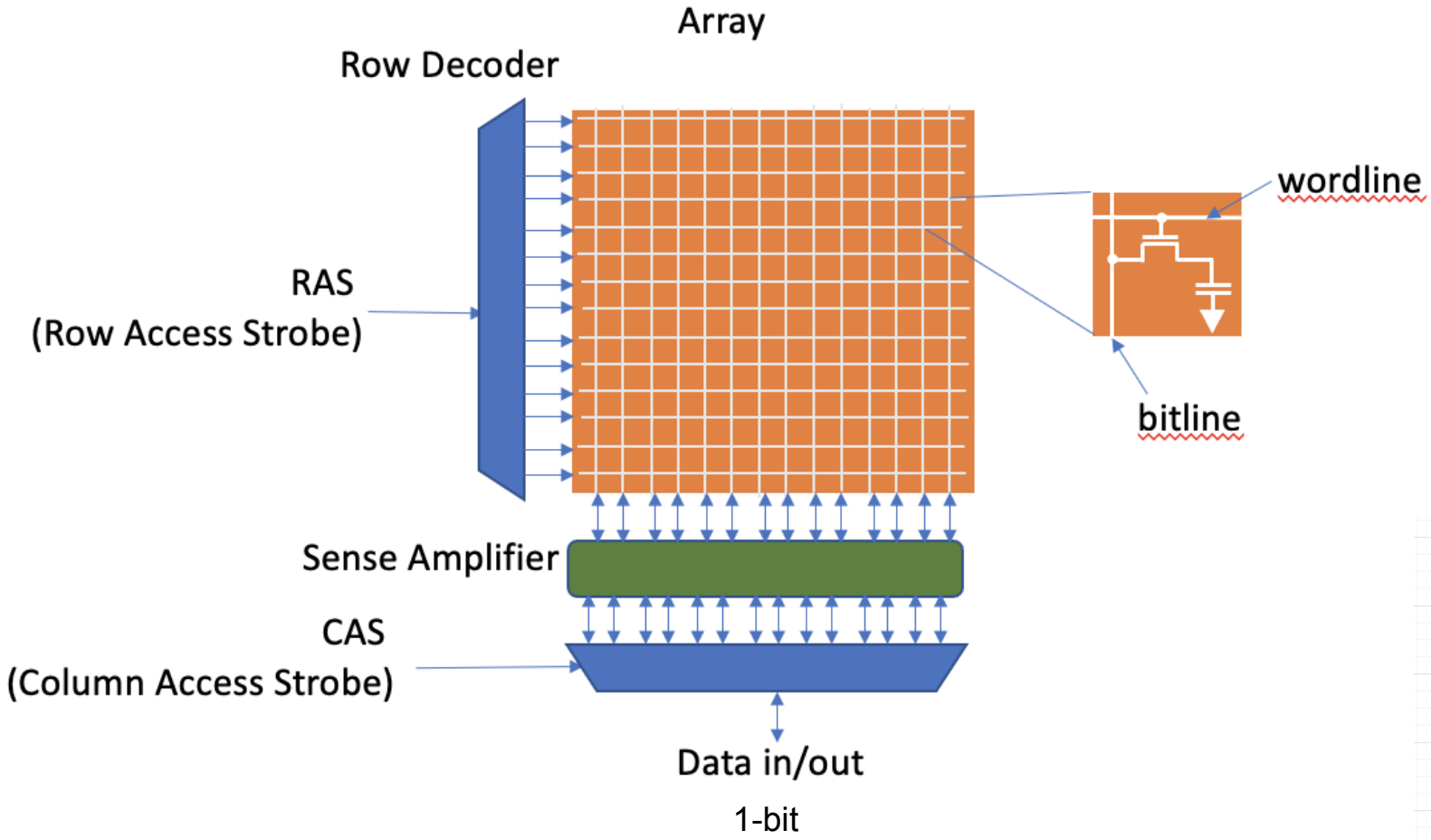


Bank 0

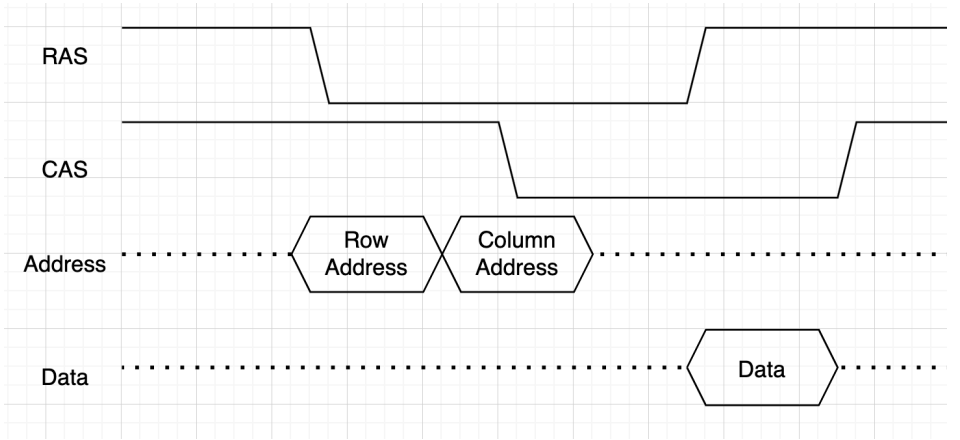


Each array provides a single bit output. Thus each bank here provides 1 byte at any given time

DRAM Organization
Basic functionality



1. Row address sent through RAS. One row is activated. Charge on the capacitor (in case of 1) flows into the sense amplifier.
2. Column address is sent through CAS to read out a bit from the specific column.
3. Row is closed: The change on the sense amplifier is fed back to the corresponding row, so as to restore the charge on the capacitor.

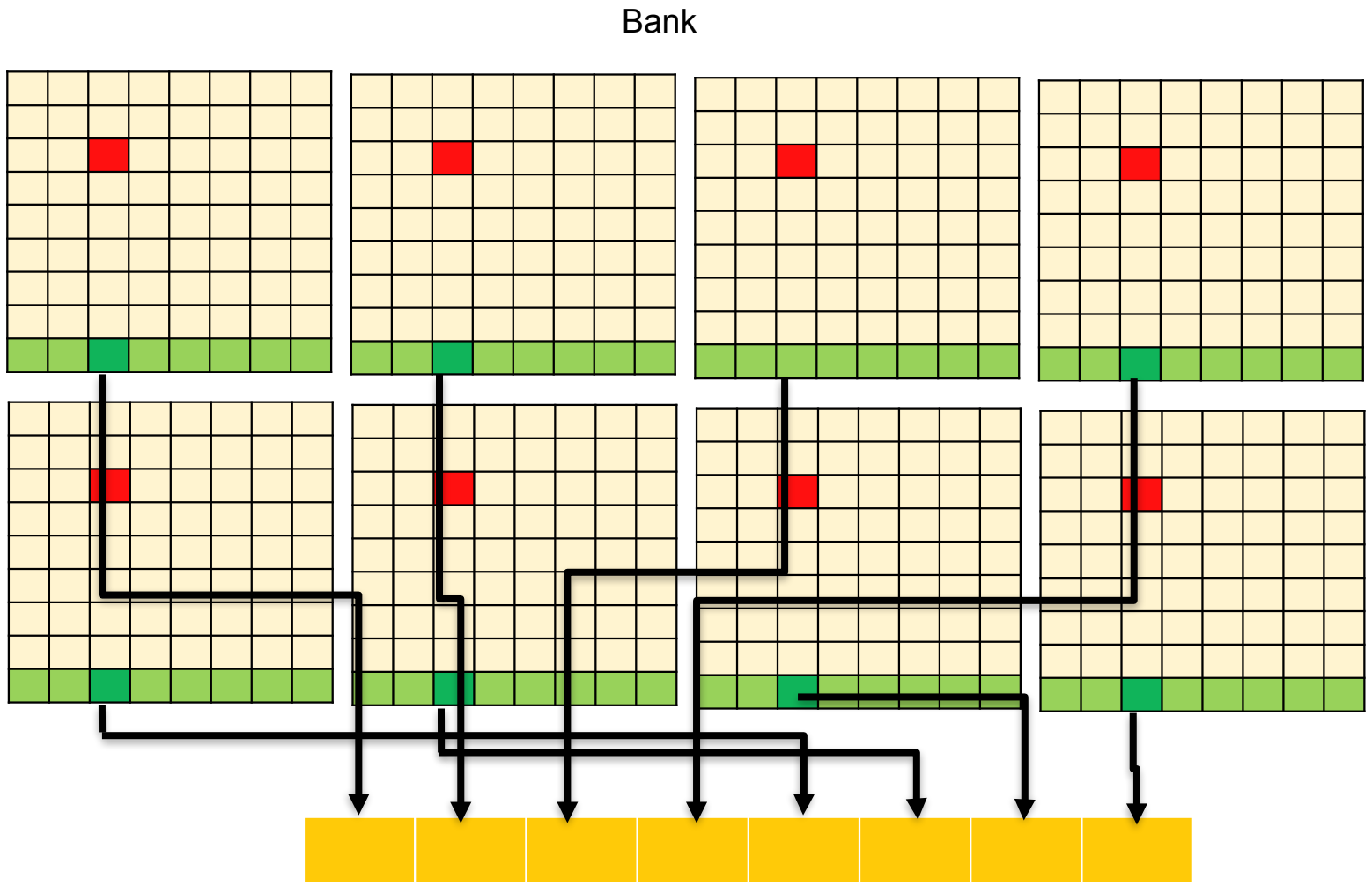


Step 1 is the slowest. Multiple banks simultaneously active to read out one byte. Too inefficient.

To read a byte

RAS: 02
CAS: 02

1-bit read from each array
to form 1-byte

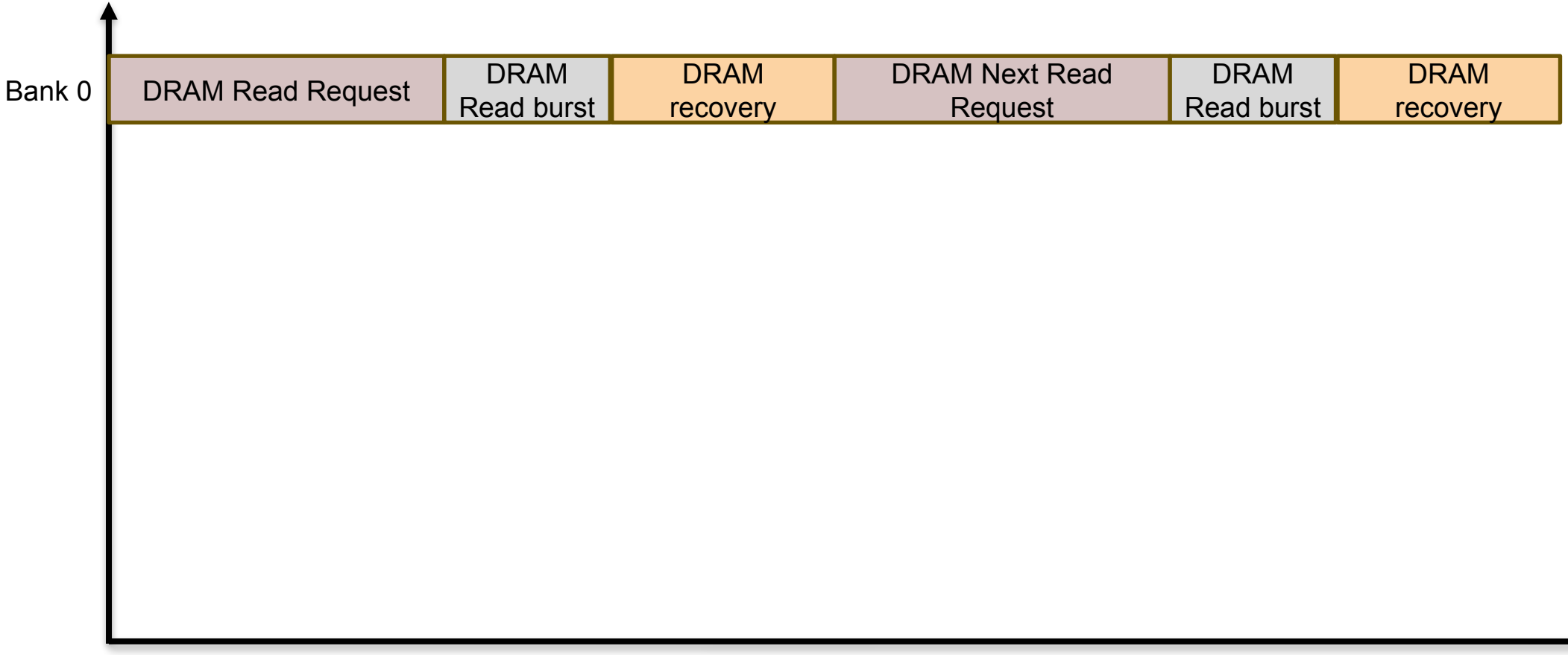


Burst Mode

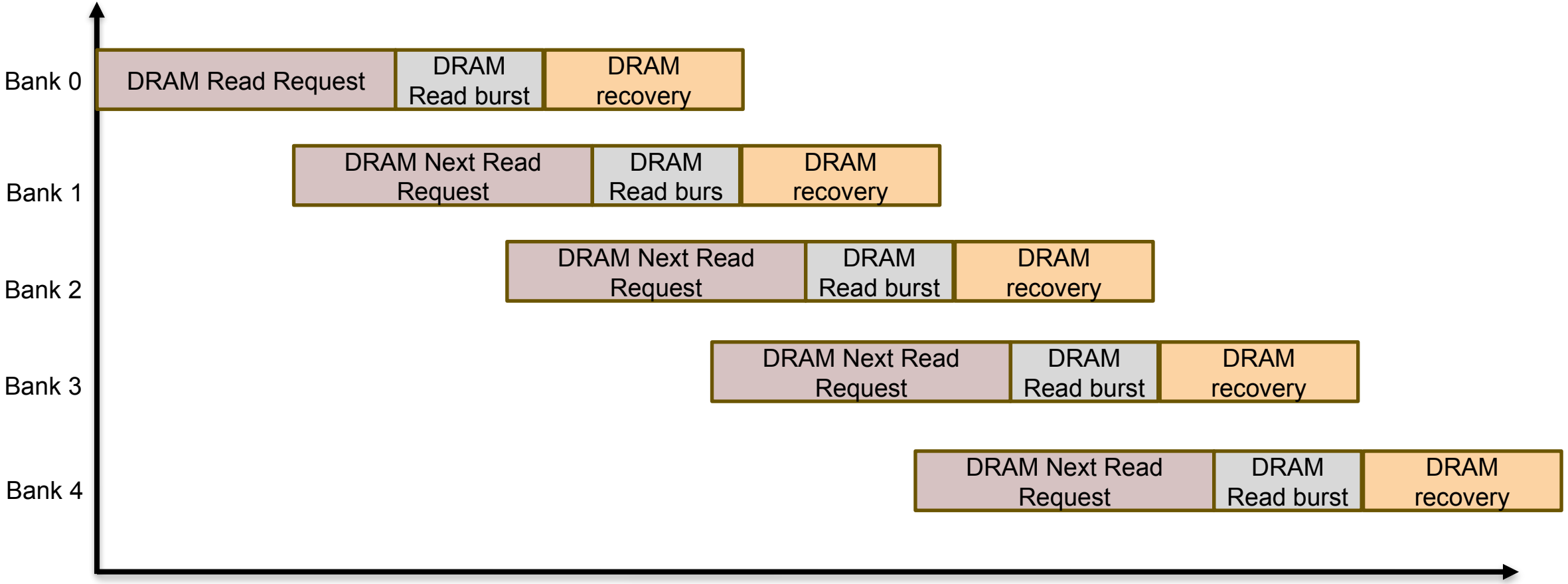
1. Row address sent through RAS. One row is activated. Charge on the capacitor (in case of 1) flows into the sense amplifier.
2. Column address is sent through CAS to read out a byte from the specific column.
3. **Read multiple columns out and stored in a buffer.**
4. Row is closed.

1. DDR2 burst size is 4 bytes
2. DDR3 burst size is 8 bytes
3. DDR4 burst size is 16 bytes

DRAM access over time



Bank Interleaving

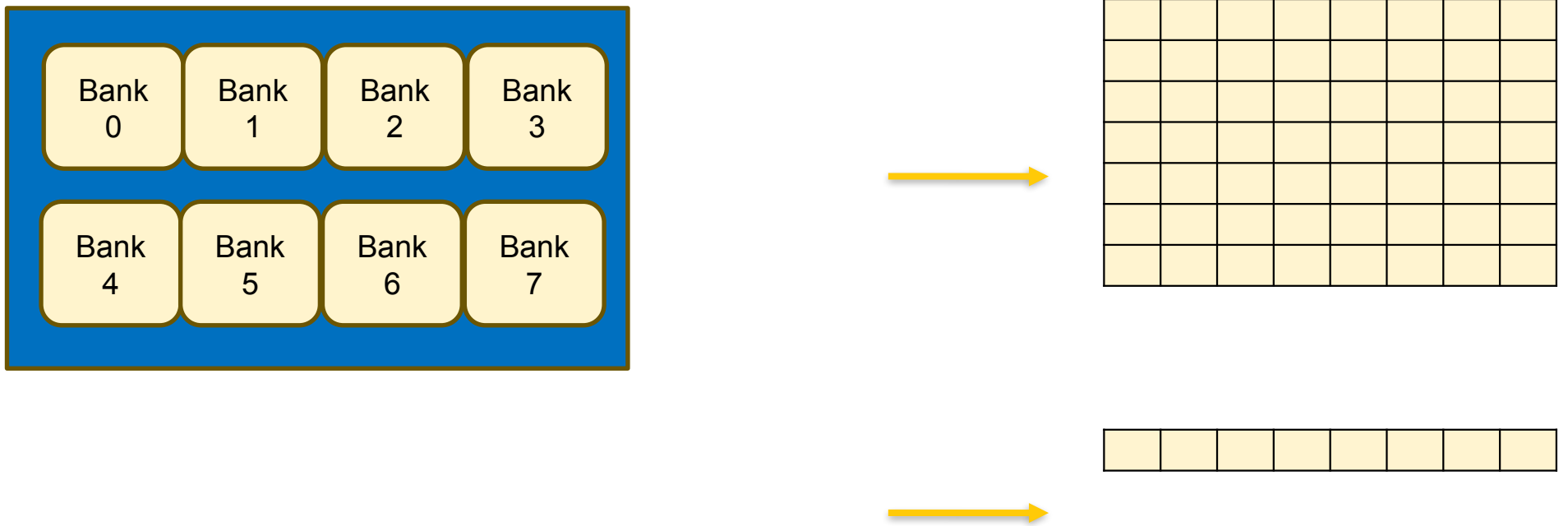


Bank Interleaving

- Each bank operates independently of other banks in the DRAM chip.
- While one bank is busy sending out data to the row buffer, the other banks can be in preparation and getting ready
- Thus, data from the dram chip can be pumped out in quick succession. Increasing throughput.
  - Data bus is always saturated and not idle

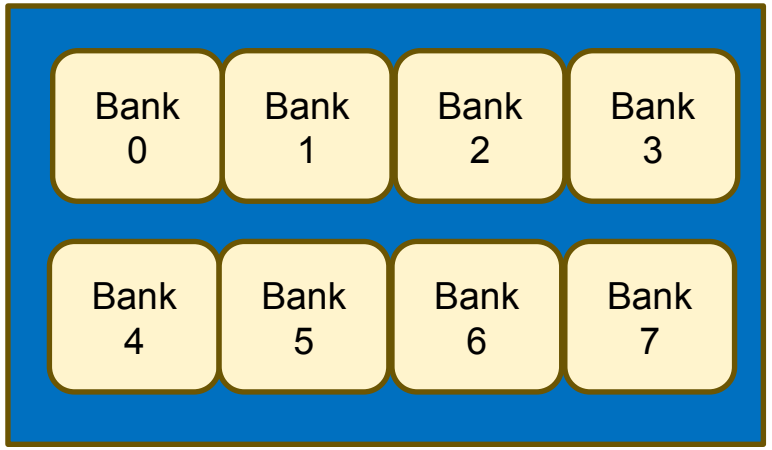
DRAM Memory Addressing

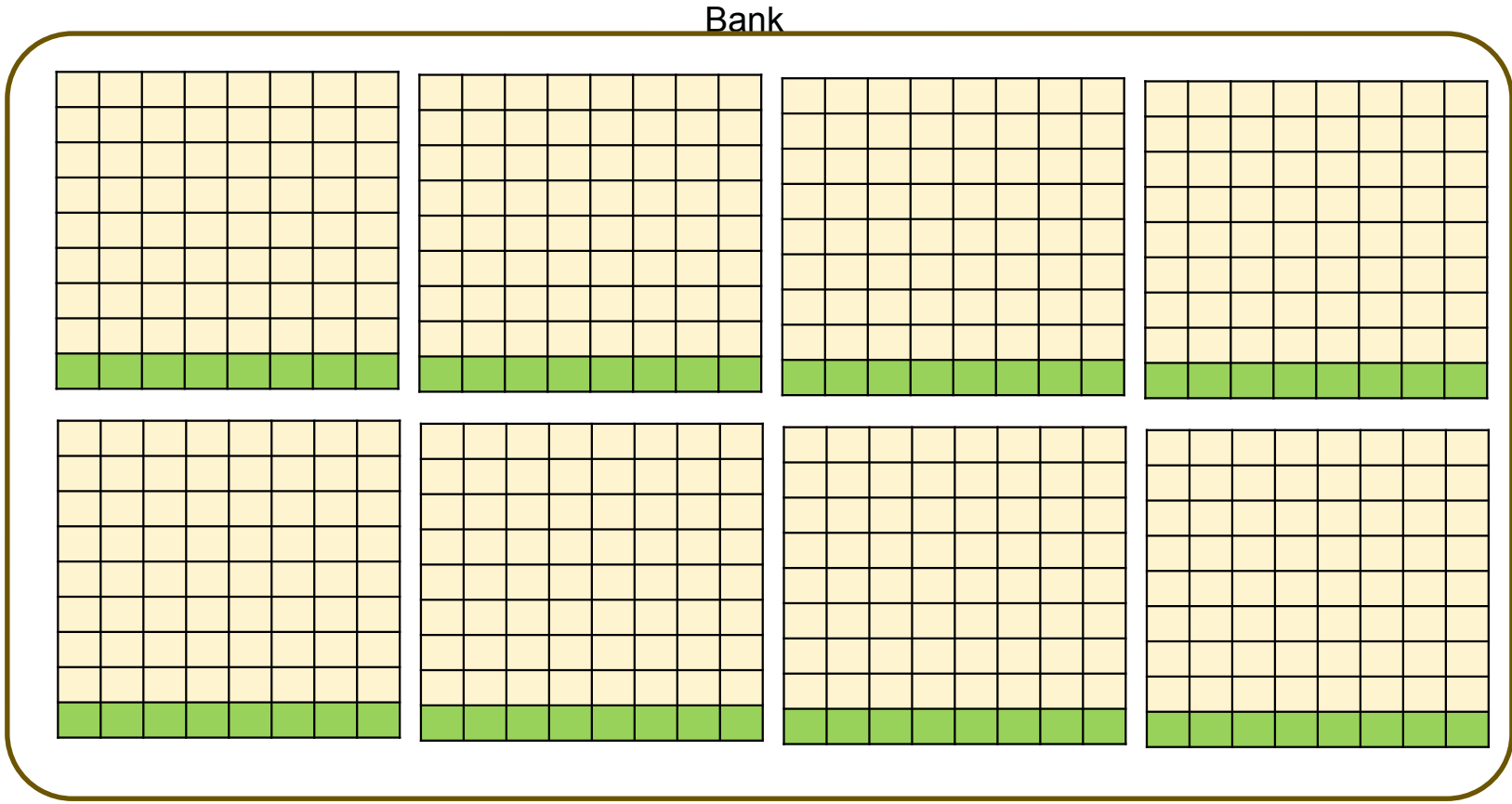
- Consider that a DRAM chip has 8 rows and 8 columns in each array
- There are 8 banks in the chip, each bank has 8 arrays that work together to provide 1 byte of data



DRAM Memory Addressing

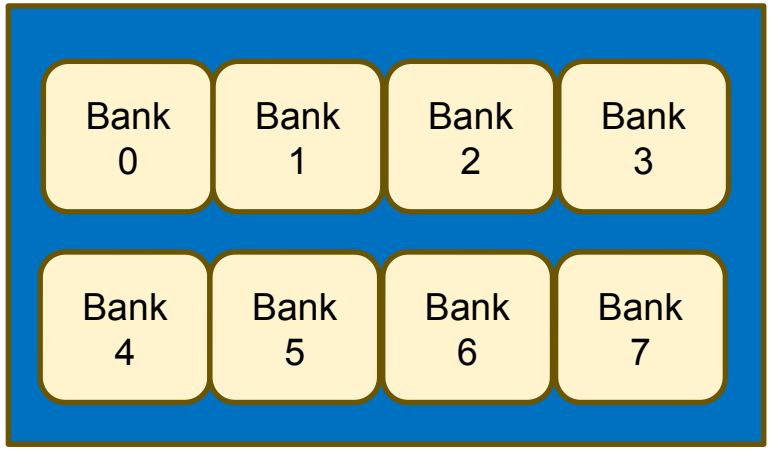
- Consider that a DRAM chip has 8 rows and 8 columns in each array
- There are 8 banks in the chip, each bank has 8 arrays that work together to provide 1 byte of data

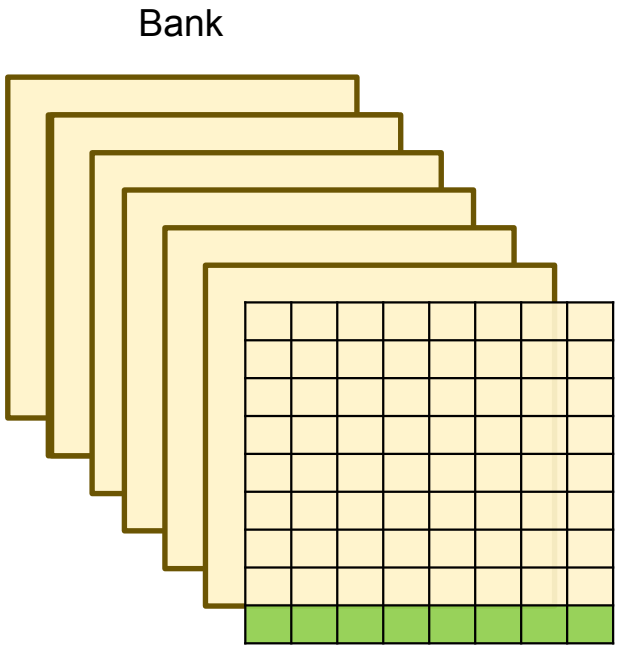




DRAM Memory Addressing

- Consider that a DRAM chip has 8 rows and 8 columns in each array
- There are 8 banks in the chip, each bank has 8 arrays that work together to provide 1 byte of data

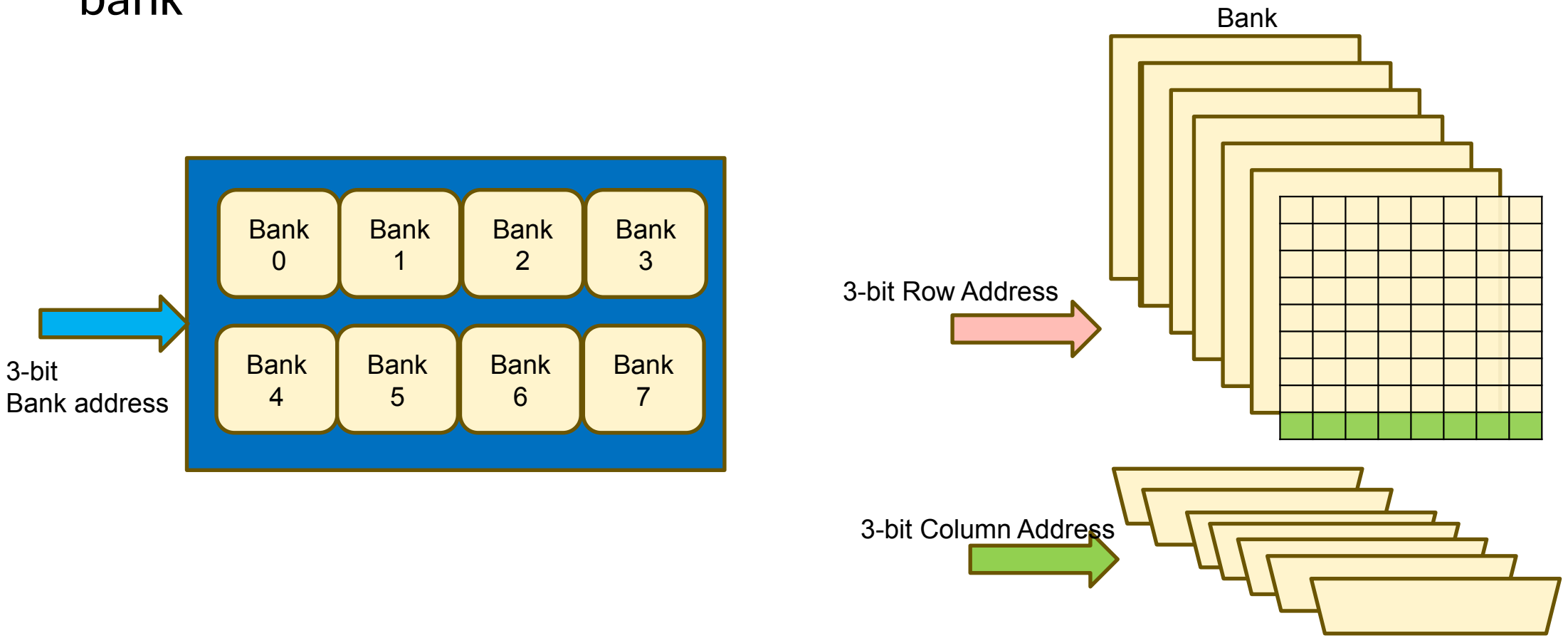




DRAM Memory Addressing

To address a byte in this DRAM chip, we need 9-bits

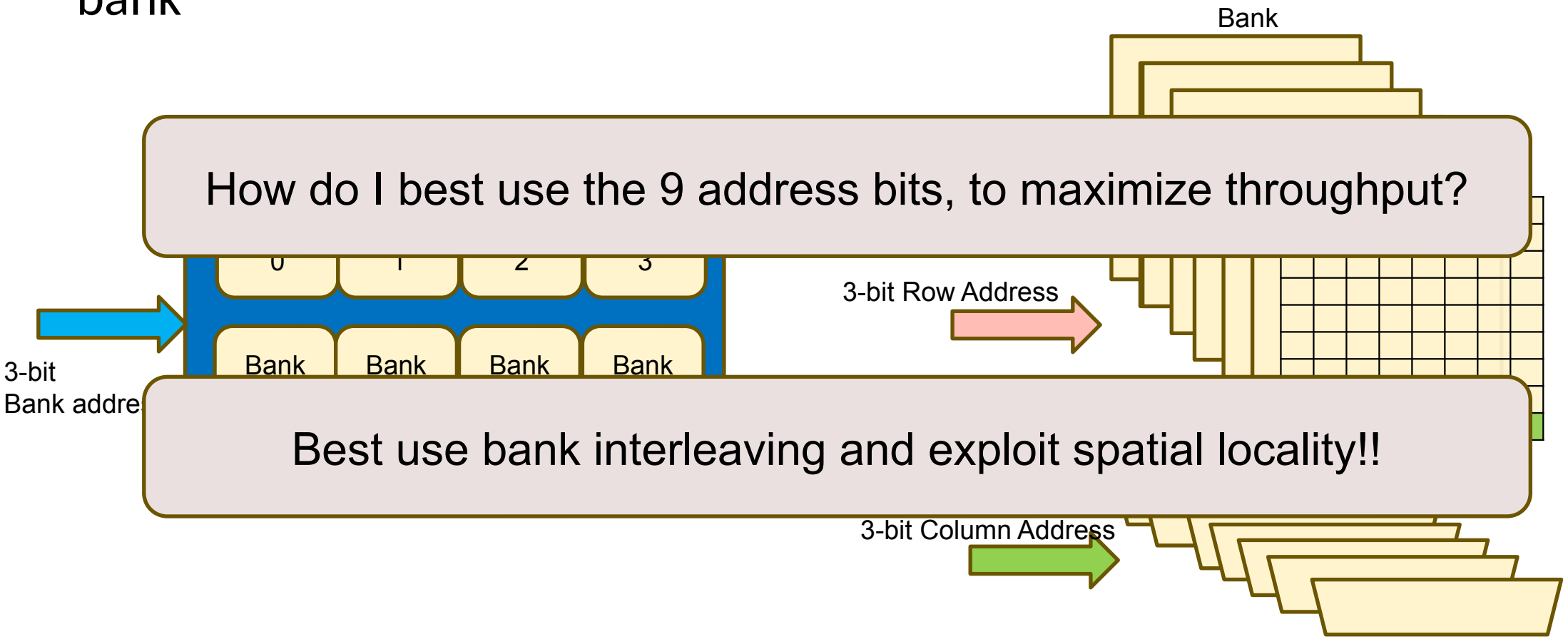
3-bits for row address; 3-bits for column address; 3-bits to select the bank



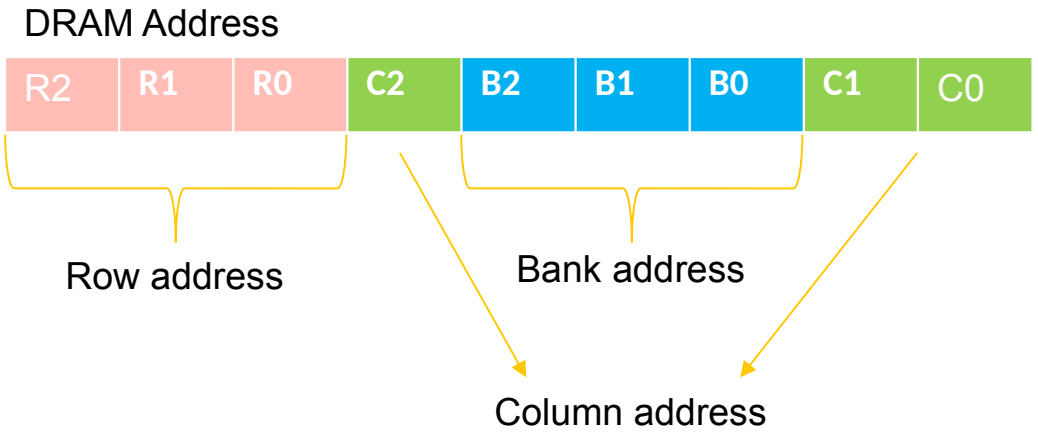
DRAM Memory Addressing

To address a byte in this DRAM chip, we need 9-bits

3-bits for row address; 3-bits for column address; 3-bits to select the bank

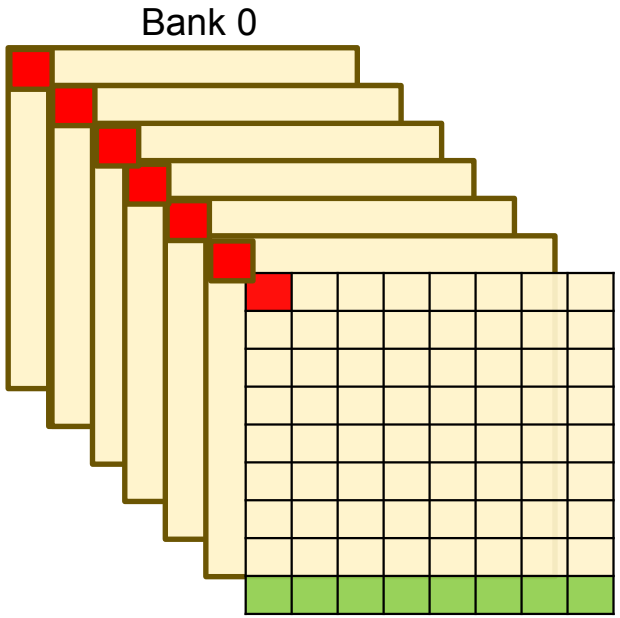


DRAM Addressing



DRAM Address

000000000 arr[0]	B0; R0; C0




```
typedef unsigned char u8;

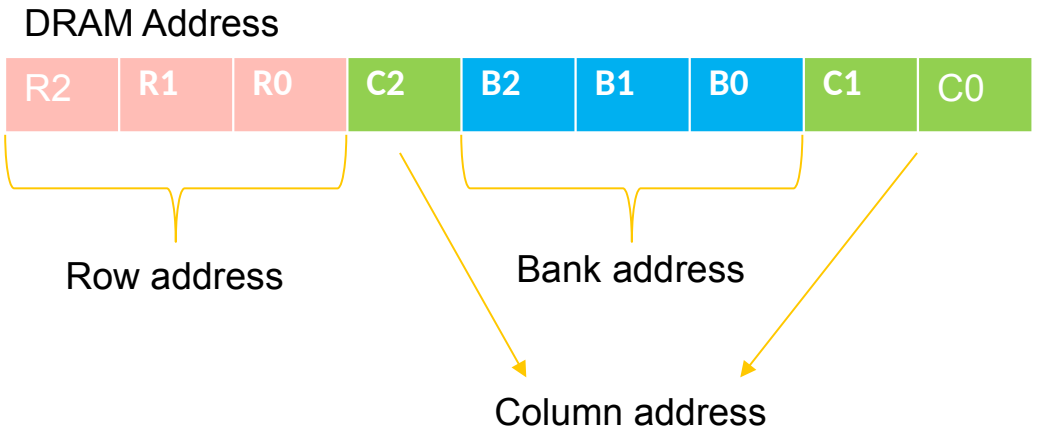
u8 arr[512];

int sumarray(){
   int x=0;
   int i;

   for(i=0; i<512; ++i)
      x += arr[i];

   return x;
}
```

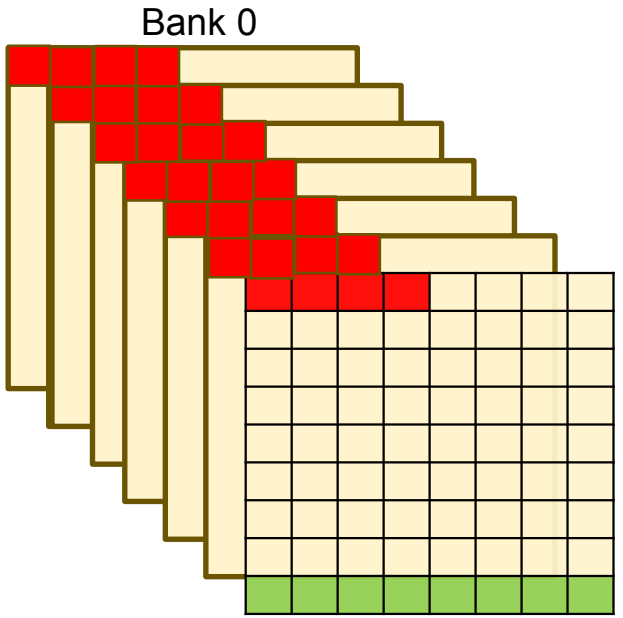
DRAM Addressing: burst mode



DRAM Address

000000000 arr[0]	B0; R0; C0
000000001 arr[1]	B0; R0; C1
000000010 arr[2]	B0; R0; C2
000000011 arr[3]	B0; R0; C3

Burst




```
typedef unsigned char u8;

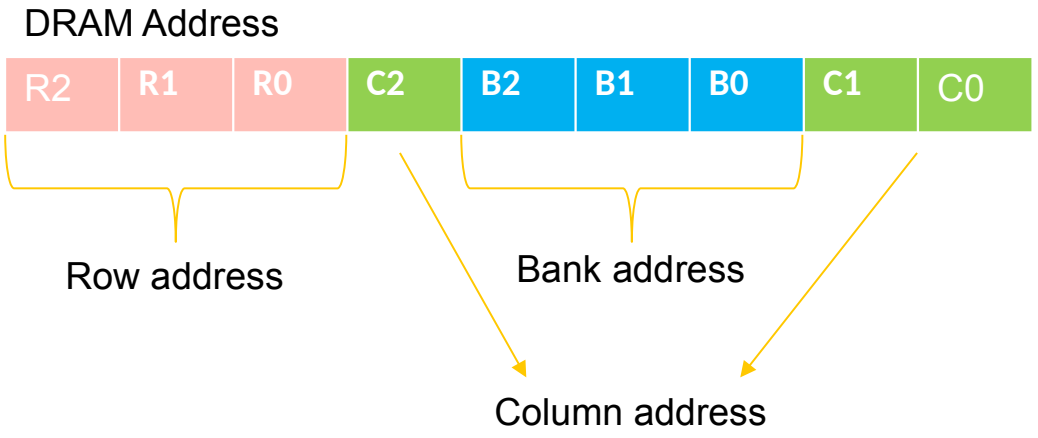
u8 arr[512];

int sumarray(){
   int x=0;
   int i;

   for(i=0; i<512; ++i)
      x += arr[i];

   return x;
}
```

DRAM Addressing: interleaving




```
typedef unsigned char u8;

u8 arr[512];

int sumarray(){
   int x=0;
   int i;

   for(i=0; i<512; ++i)
      x += arr[i];

   return x;
}
```

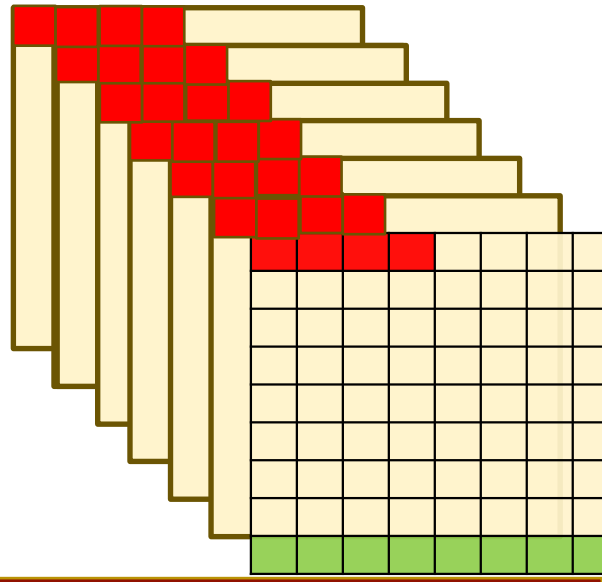
DRAM Address

000000000	B0; R0; C0
000000001	B0; R0; C1
000000010	B0; R0; C2
000000011	B0; R0; C3
000000000	B1; R0; C0
000000001	B1; R0; C1
000000010	B1; R0; C2
000000011	B1; R0; C3

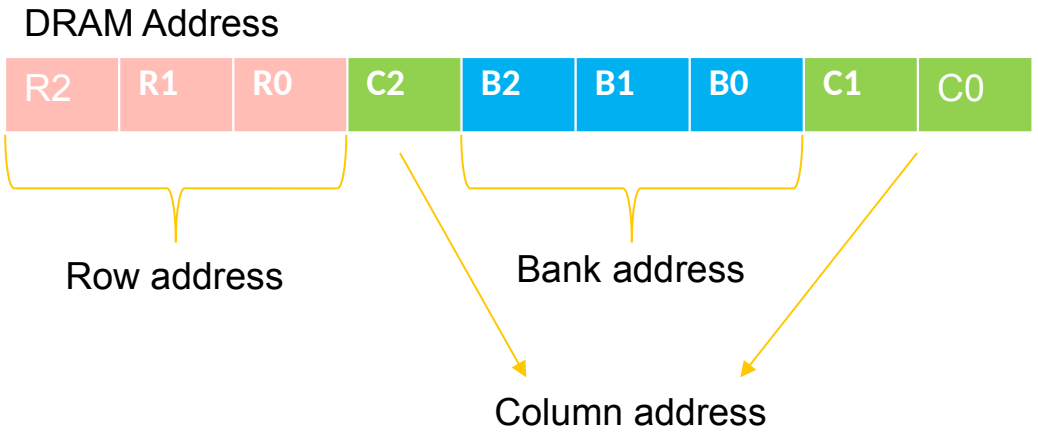
Bank 0 recovering

Bank 1 transmitting

Bank 1



DRAM Addressing: interleaving




```
typedef unsigned char u8;

u8 arr[512];

int sumarray(){
   int x=0;
   int i;

   for(i=0; i<512; ++i)
      x += arr[i];

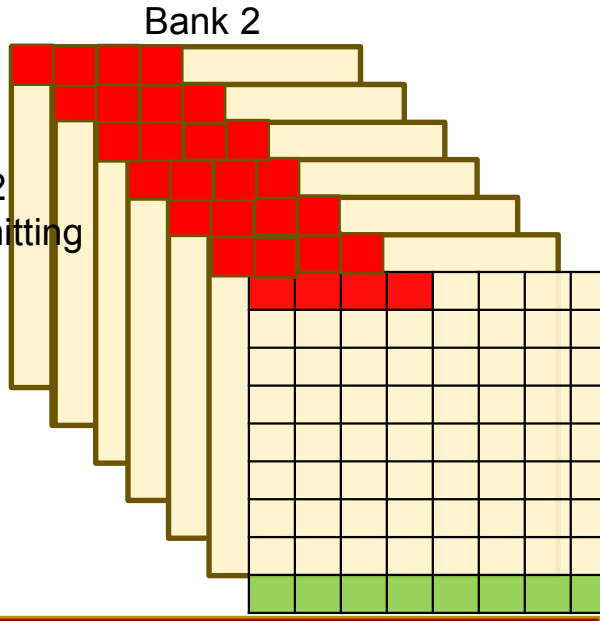
   return x;
}
```

DRAM Address	
000000000	B0; R0; C0
000000001	B0; R0; C1
000000010	B0; R0; C2
000000011	B0; R0; C3
000000000	B1; R0; C0
000000001	B1; R0; C1
000000010	B1; R0; C2
000000011	B1; R0; C3
000000000	B2; R0; C0
000000001	B2; R0; C1
000000010	B2; R0; C2
000000011	B2; R0; C3

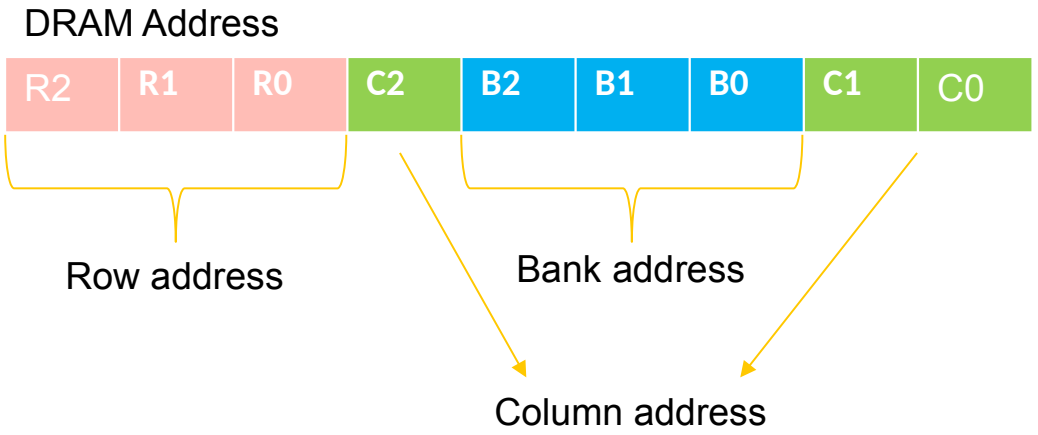
Bank 0 recovering

Bank 1 recovering

Bank 2 transmitting



DRAM Addressing: interleaving




```
typedef unsigned char u8;

u8 arr[512];

int sumarray(){
   int x=0;
   int i;

   for(i=0; i<512; ++i)
      x += arr[i];

   return x;
}
```

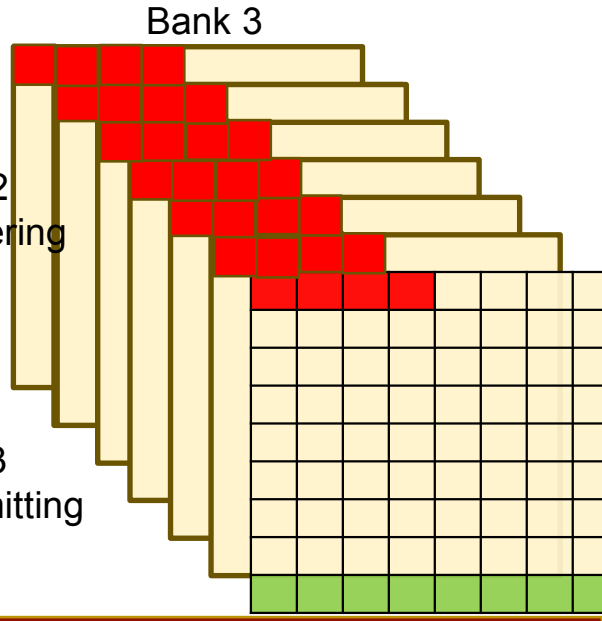
DRAM Address	
000000000	B0; R0; C0
000000001	B0; R0; C1
000000010	B0; R0; C2
000000011	B0; R0; C3
000000100	B1; R0; C0
000000101	B1; R0; C1
000000110	B1; R0; C2
000000111	B1; R0; C3
000001000	B2; R0; C0
000001001	B2; R0; C1
000001010	B2; R0; C2
000001011	B2; R0; C3
000001100	B3; R0; C0
000001110	B3; R0; C1

Bank 0 recovering

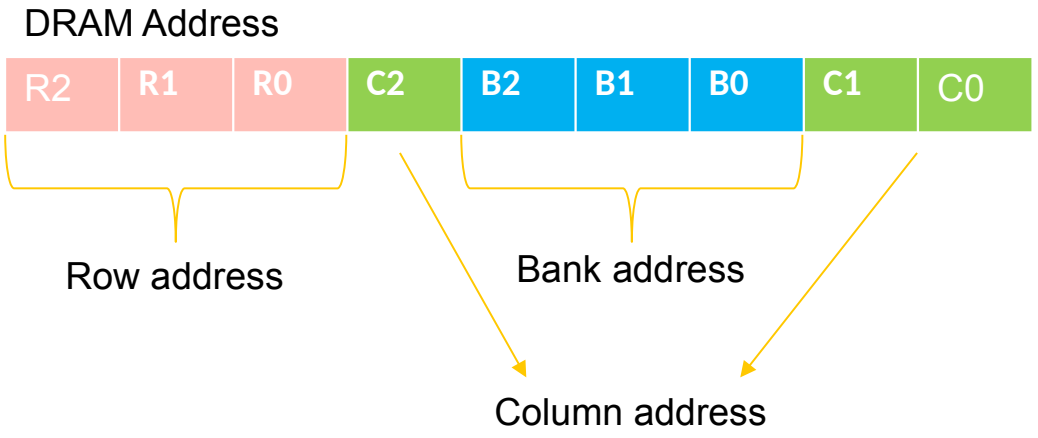
Bank 1 recovering

Bank 2 recovering

Bank 3 transmitting



DRAM Addressing: interleaving



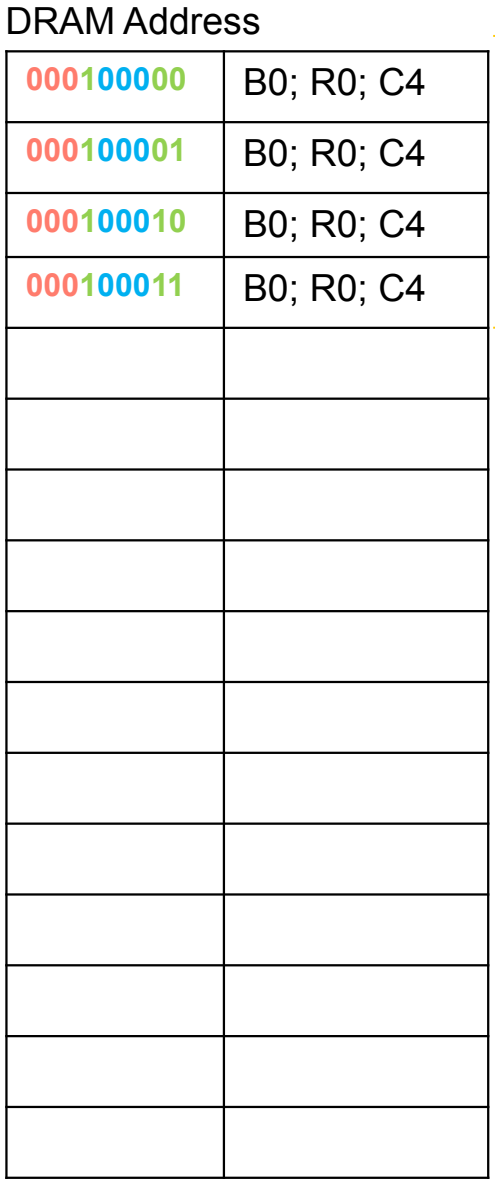

```
typedef unsigned char u8;

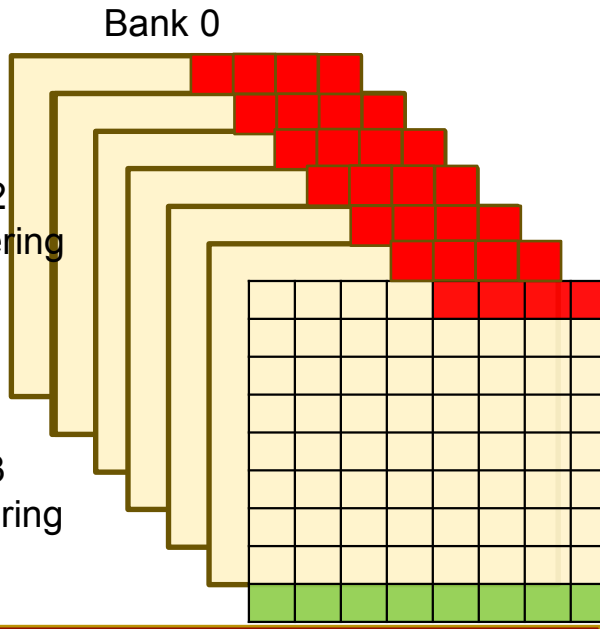
u8 arr[512];

int sumarray(){
   int x=0;
   int i;

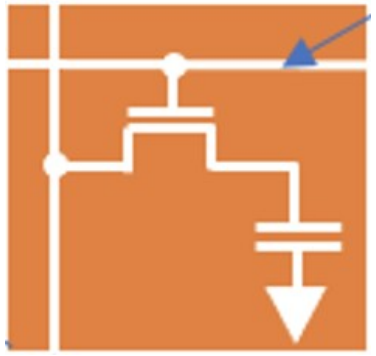
   for(i=0; i<512; ++i)
      x += arr[i];

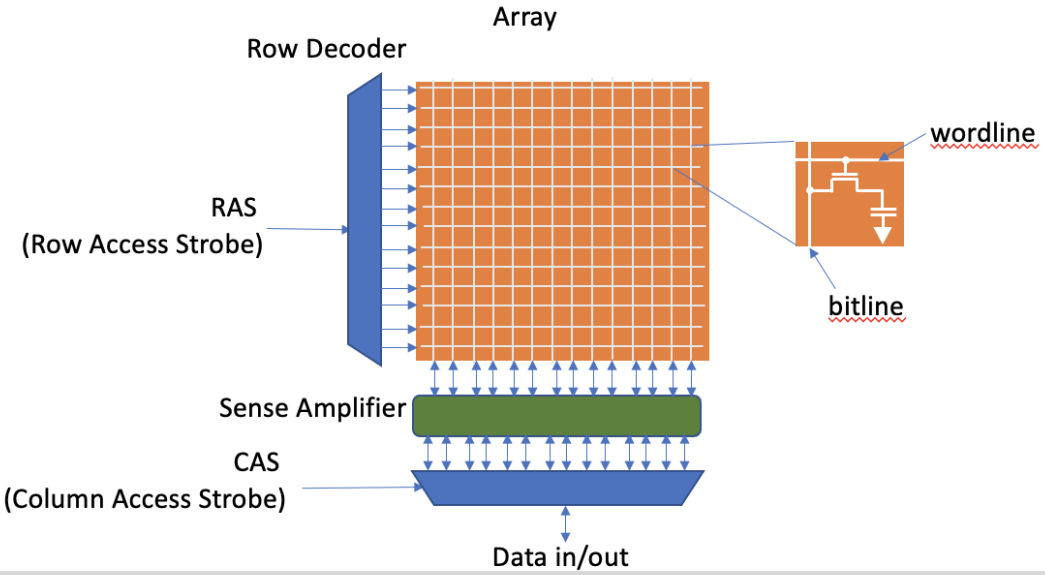
   return x;
}
```





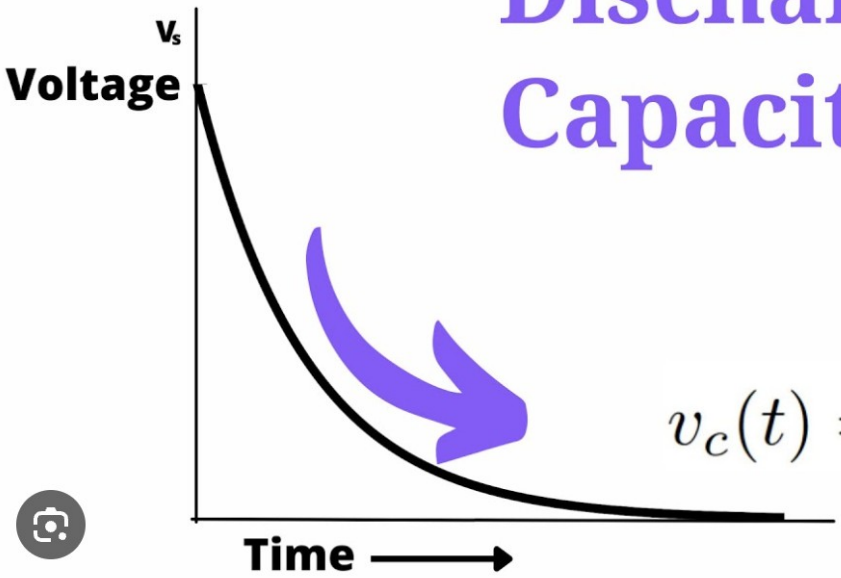
DRAM Refreshing





To refresh, need to read a row and write it back to recharge the capacitor

Discharging Capacitor



$$v_c(t) = V_s e^{\frac{-t}{RC}}$$

Capacitor discharges over a period of time due to leakage
Thus they need to be refreshed periodically

DRAM Refresh: Done by memory controller

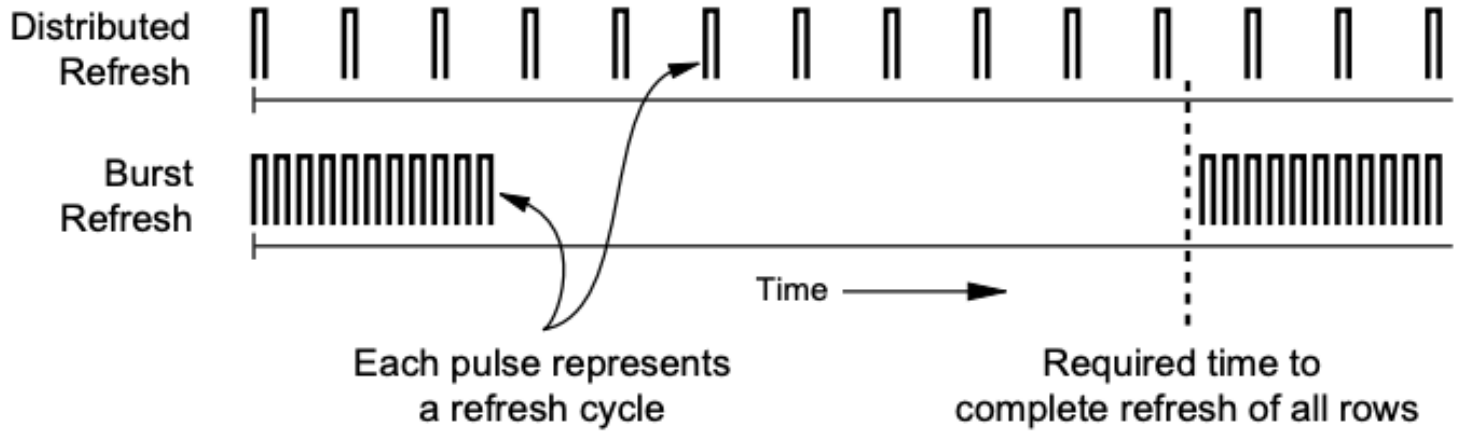
Distributed Refresh

Refresh is evenly spaced.
For example, if refresh time is 64ms and #rows=4096,
A refresh is done every 15.6us,

Burst Refresh

Perform a series of refreshes one after the other until entire bank is refreshed

For example, if time taken for one refresh is 130ns and #rows=4096, then, 0.5324ms spent on refresh.
If Refresh time is 64ms, useful period is 64-0.5324=63.468ms.



DRAM and the Mantras

”Exploit the program’s Spatial Locality”

”Exploit the program’s Temporal Locality”

”Parallelize to speed up execution”

”Leave no hardware module unused”



DRAM and Complications

Complications

”Physical characteristics”

”Cost and size”

”Complexity of design”

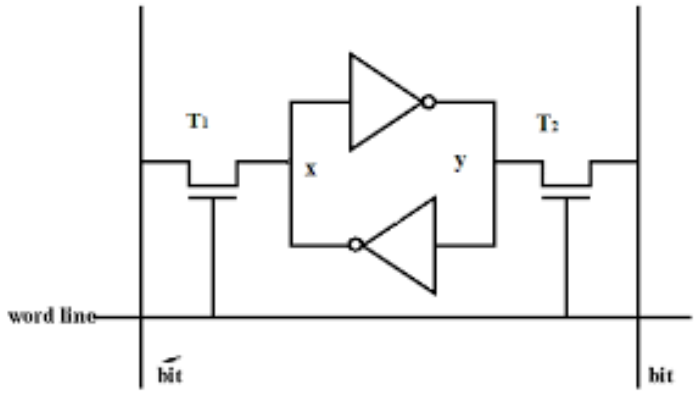


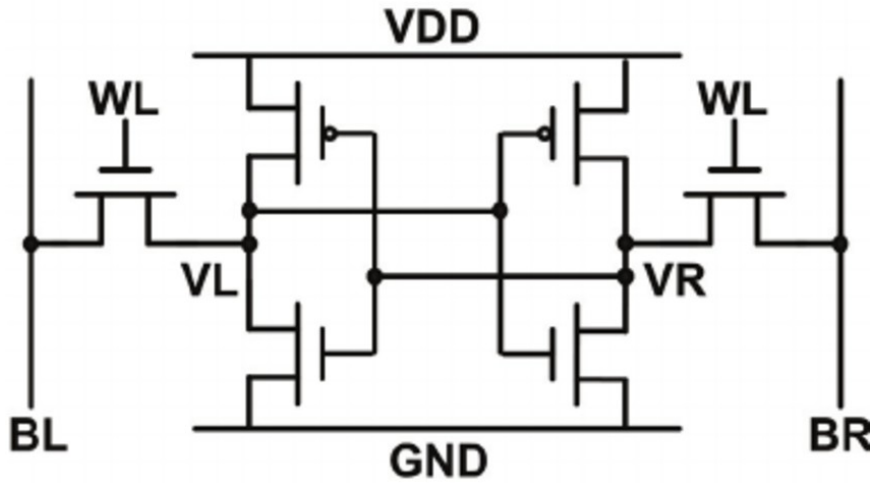
Cache Memories

Ref. Computer Organization and Design: The Hardware/Software Interface. Patterson and Hennessy

Memory Latencies

	Capacity	Latency	Cost/GB
Register	1000s of bits	20 ps	$$$$
SRAM	~10 KB-10 MB	1-10 ns	~$1000
DRAM	~10 GB	80 ns	~$10

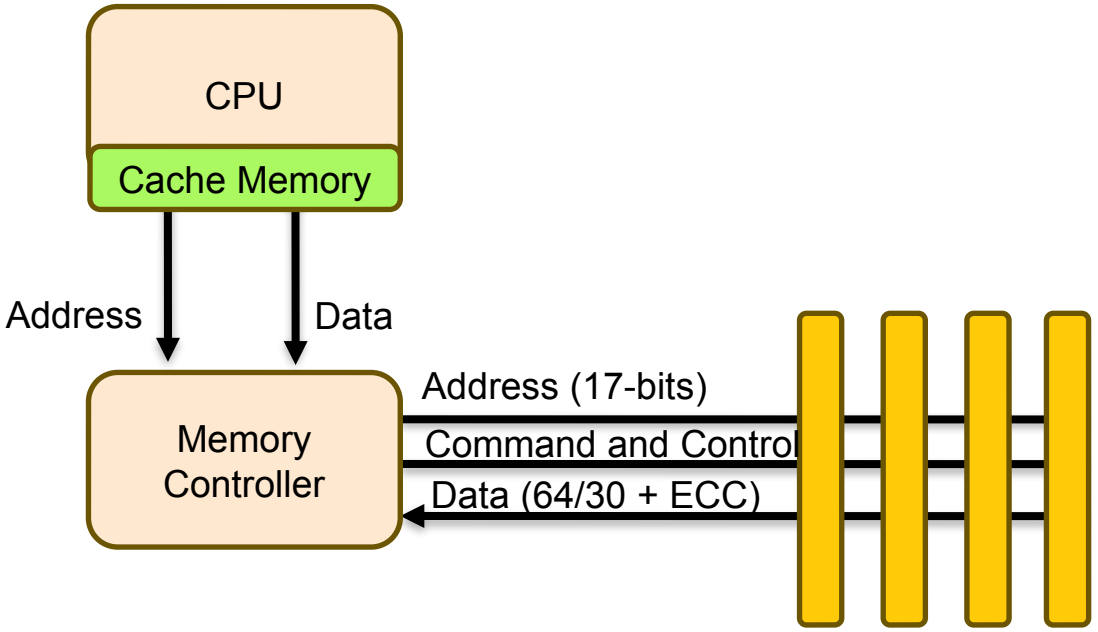




6 transistors

We can't replace DRAMs with SRAMs due to the large cost and size,
But we can have small buffers, called cache memories

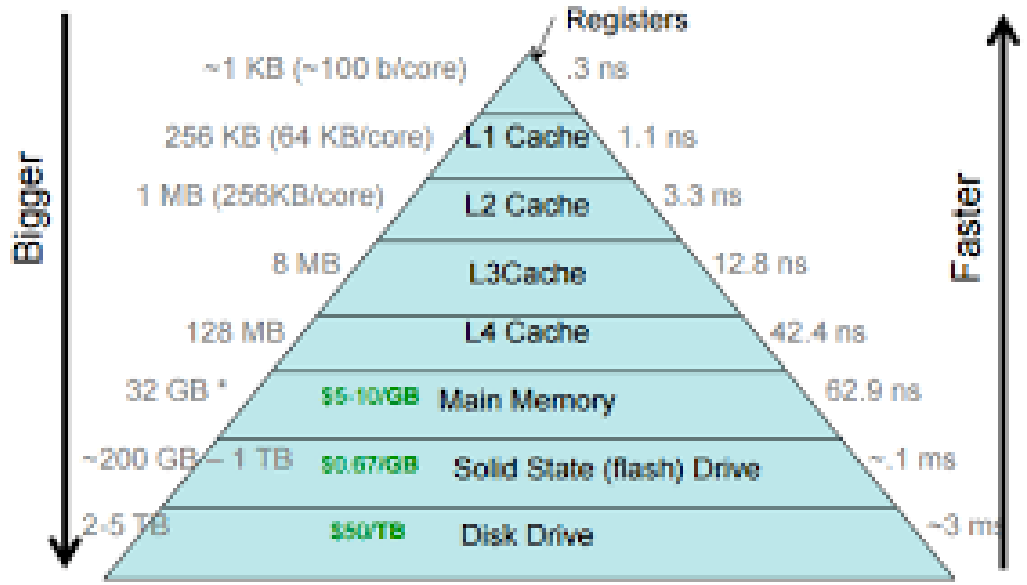
Cache Memories

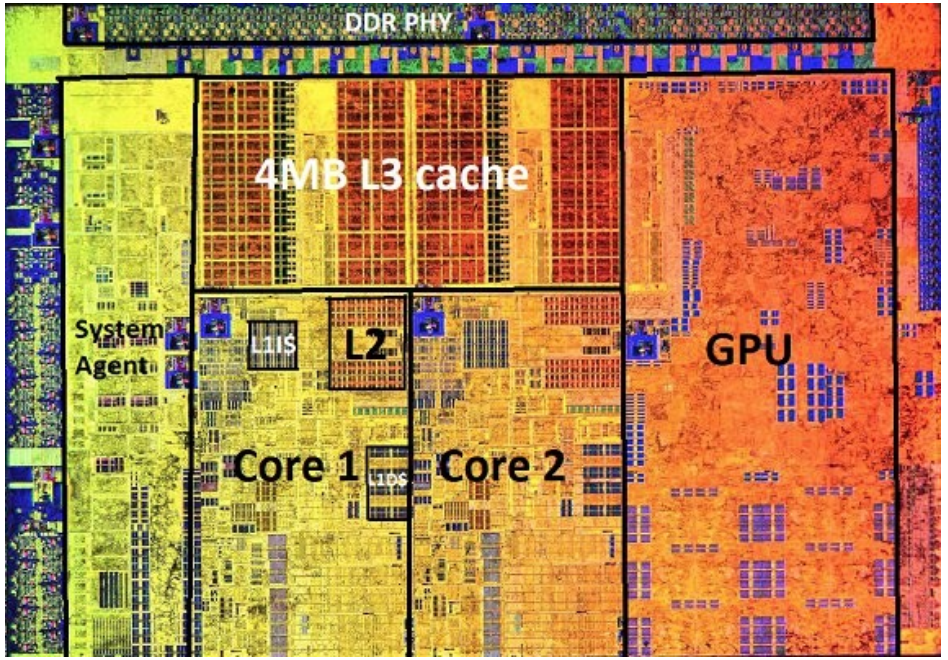


Cache Memories:

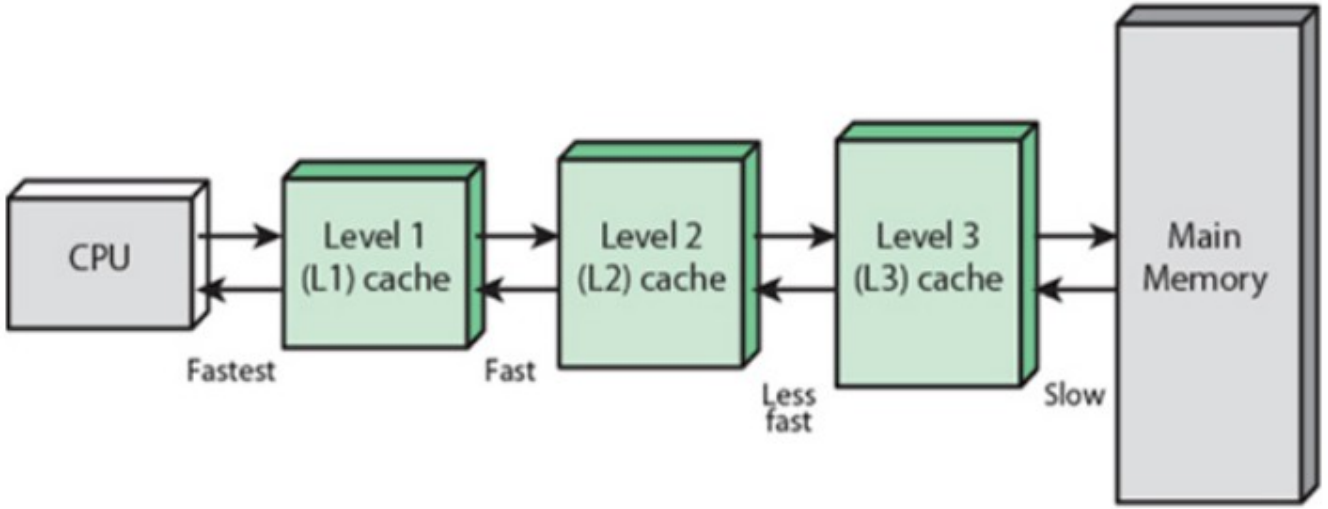
- Closely coupled with the CPU.
- Built of SRAM (instead of DRAM)
- Stores recently used instructions and data. (exploiting spatial and temporal locality)
- Small compared to DRAM size.

Hierarchy of Memories

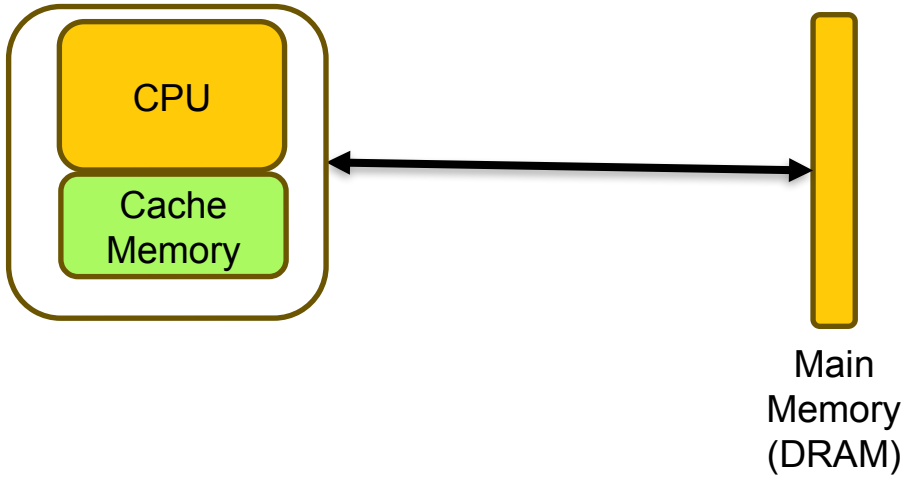




Intel Core 2 Sandybridge



Cache Memory is small compared to DRAM

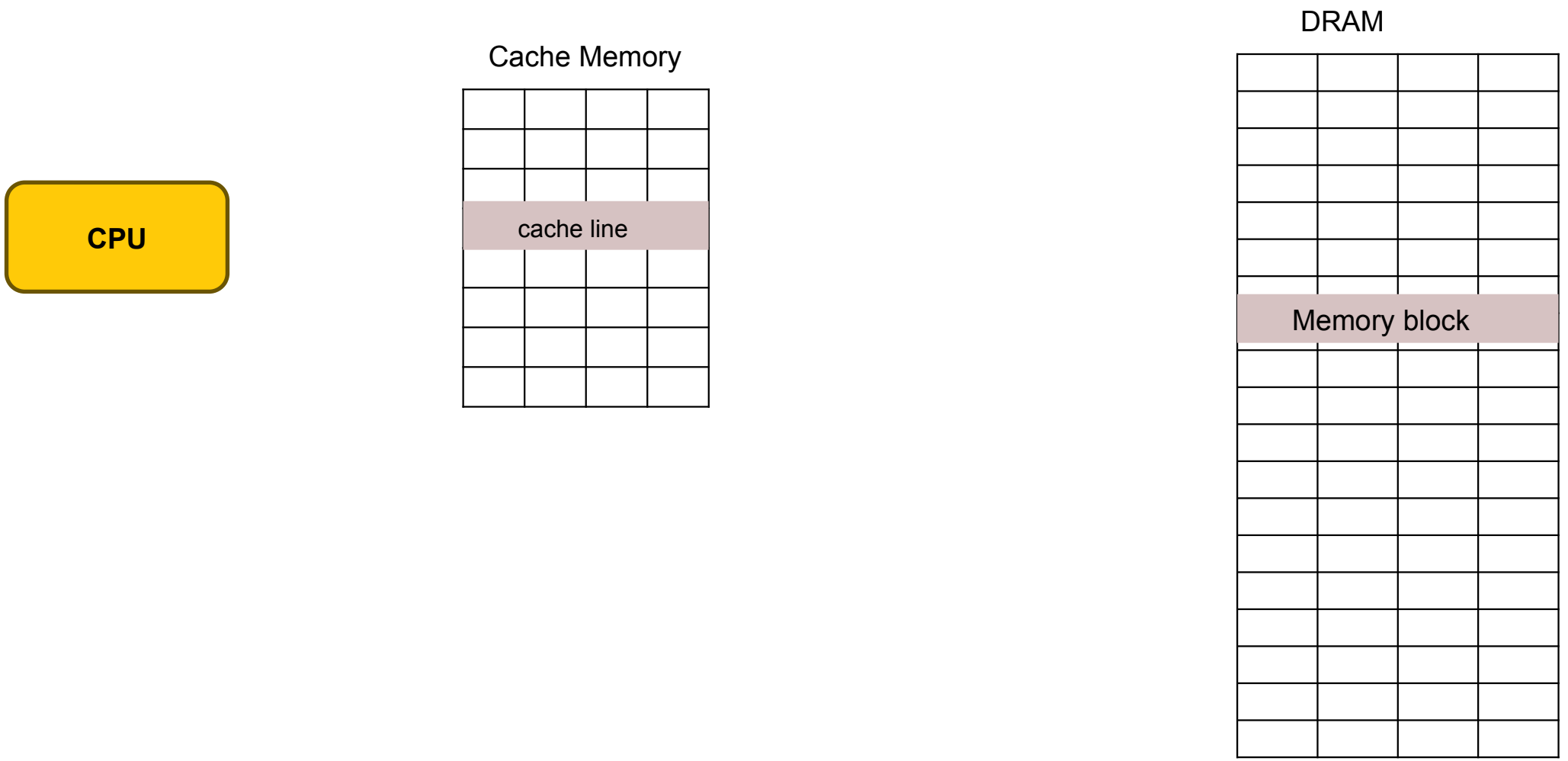


- Cache memory stores only a small subset of the DRAM
- Example: Cache memory: 32KB, DRAM 16GB
- Questions:
  - What subset of DRAM to store in cache?

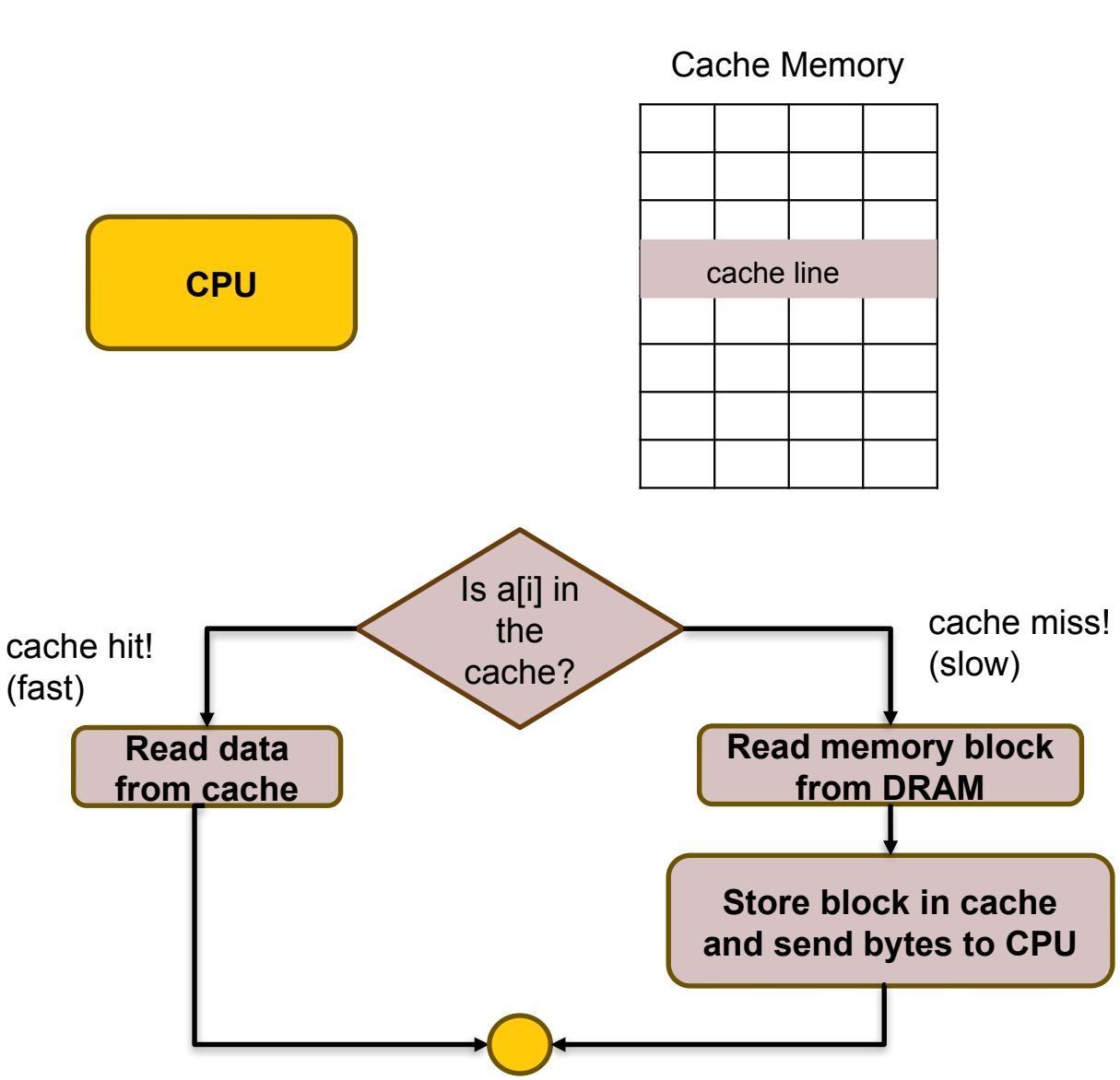
    Exploit locality
  - How to search if data/instruction is present in the cache?

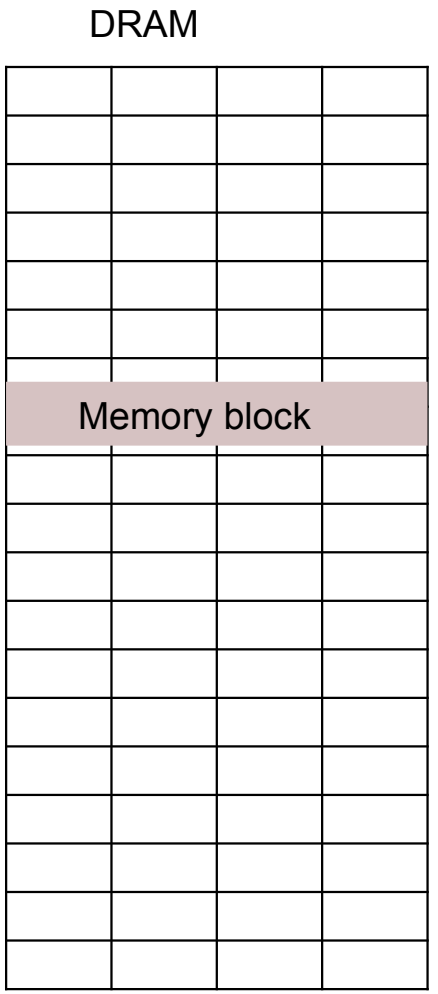
    Mapping Algorithms

How does cache memory work?



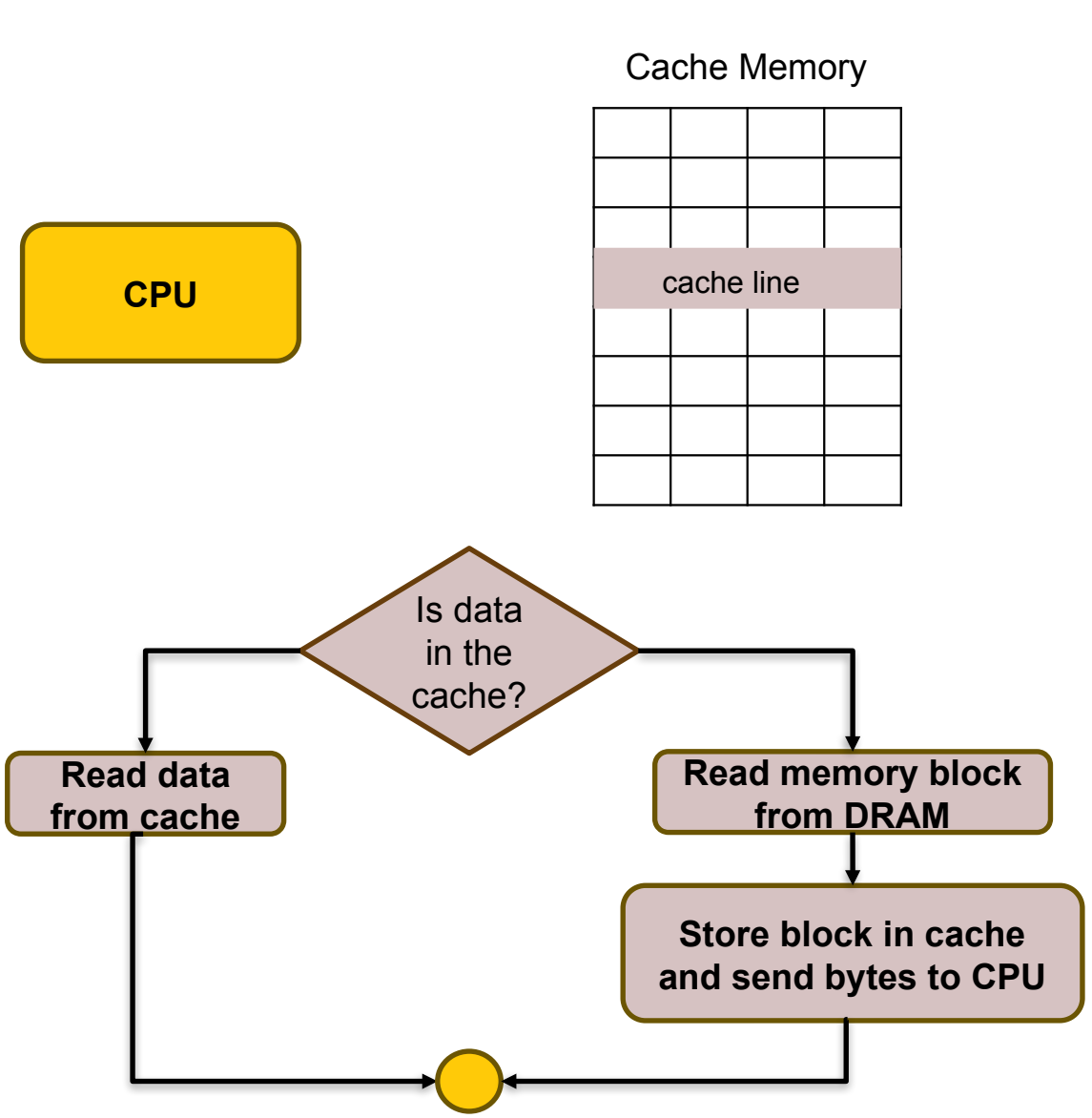
How does cache memory work?

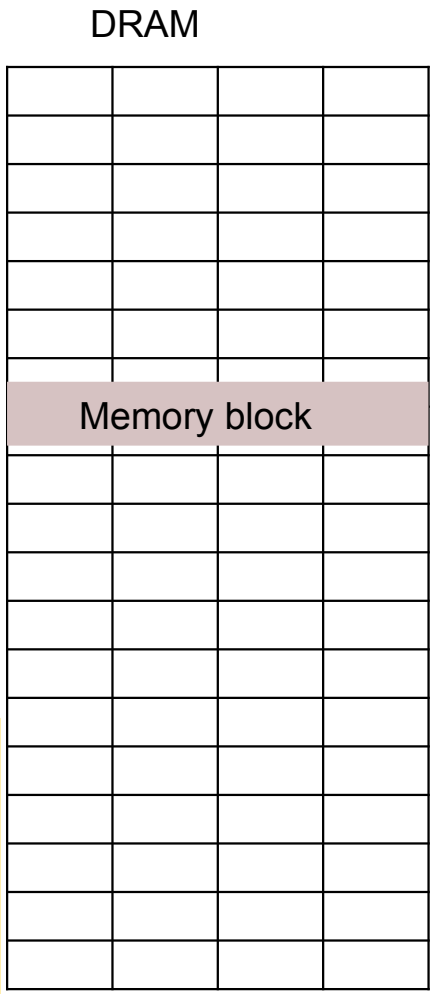




Connection between cache block and DRAM burst

How does cache memory work?






```
x = a[i];
.
.
.
y = a[i+1];
```

X = a[i] will have a cache miss; y=a[i+1] will have a cache hit!

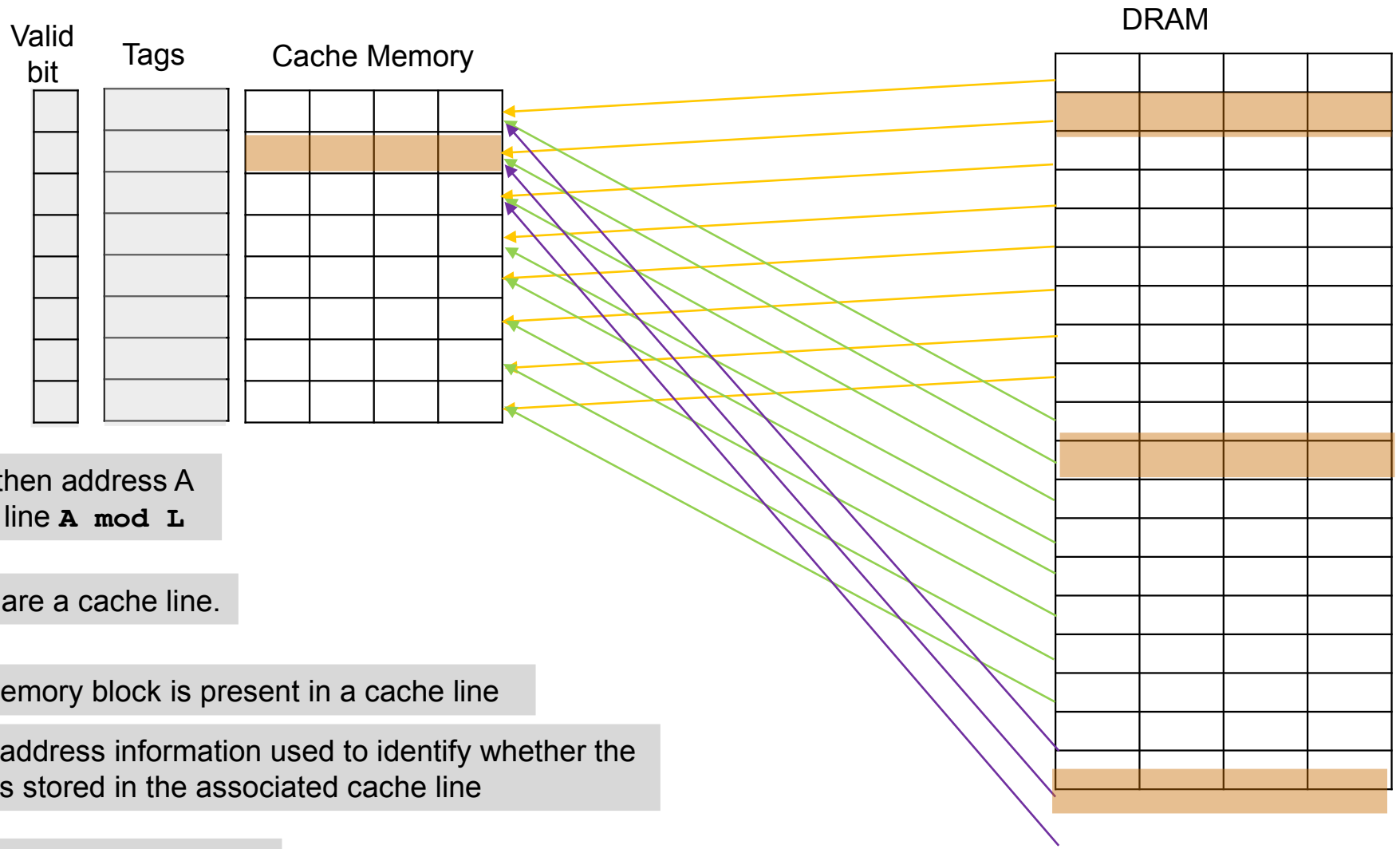
Mapping addresses in cache memory

Cache size much smaller than DRAM; Therefore mapping required

Mapping can be

- Direct mapped
- Fully associate
- Set-associative

Direct Mapped Cache Memories



If there are L cache lines, then address A gets mapped to the cache line **`A mod L`**

Multiple memory blocks share a cache line.

Tags help identify which memory block is present in a cache line

Tag: A tag contains address information used to identify whether the memory block that is stored in the associated cache line

Valid bit: Set to 1 if the tag has a valid address

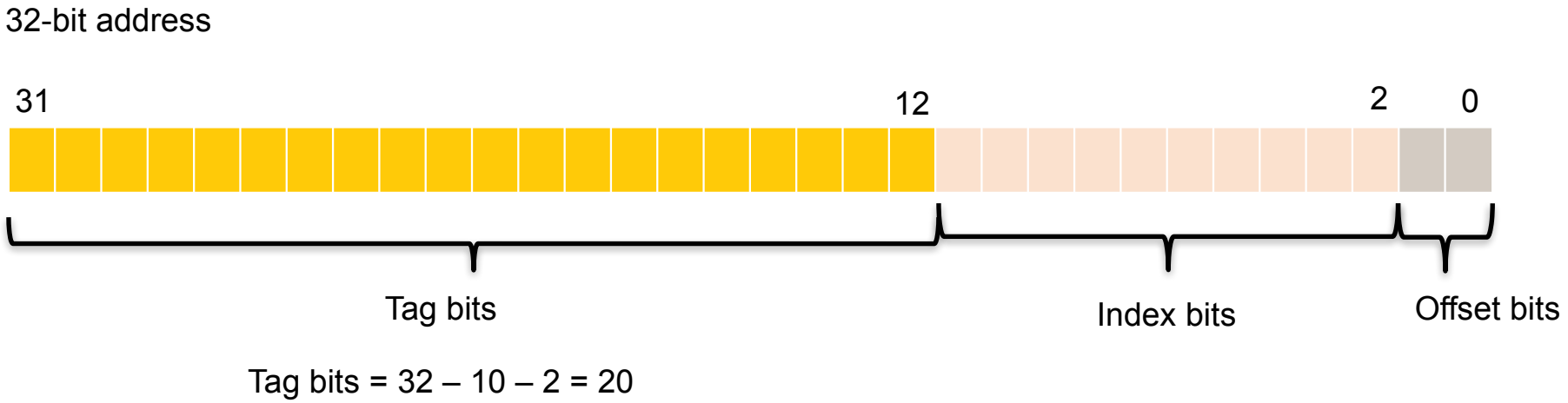
Tag and Addresses

- Example:

Direct mapped cache with 1024 cache lines and each cache line holds 4-bytes

…. $\log_2 1024 = 10$ address bits required to identify the cache line

…. $\log_2 4 = 2$ address bits required to identify the offset within the cache line



Tag bits = 32 – 10 – 2 = 20

Direct Mapped Cache Hardware View

- Cache size 1024 lines →10 index bits
- Cache line size 1 word (4 bytes) → 2 offset bits
- Address size is 64-bits → 64 – 10 – 2 = 52 bits of tag.
- Total number of bits needed by this cache is

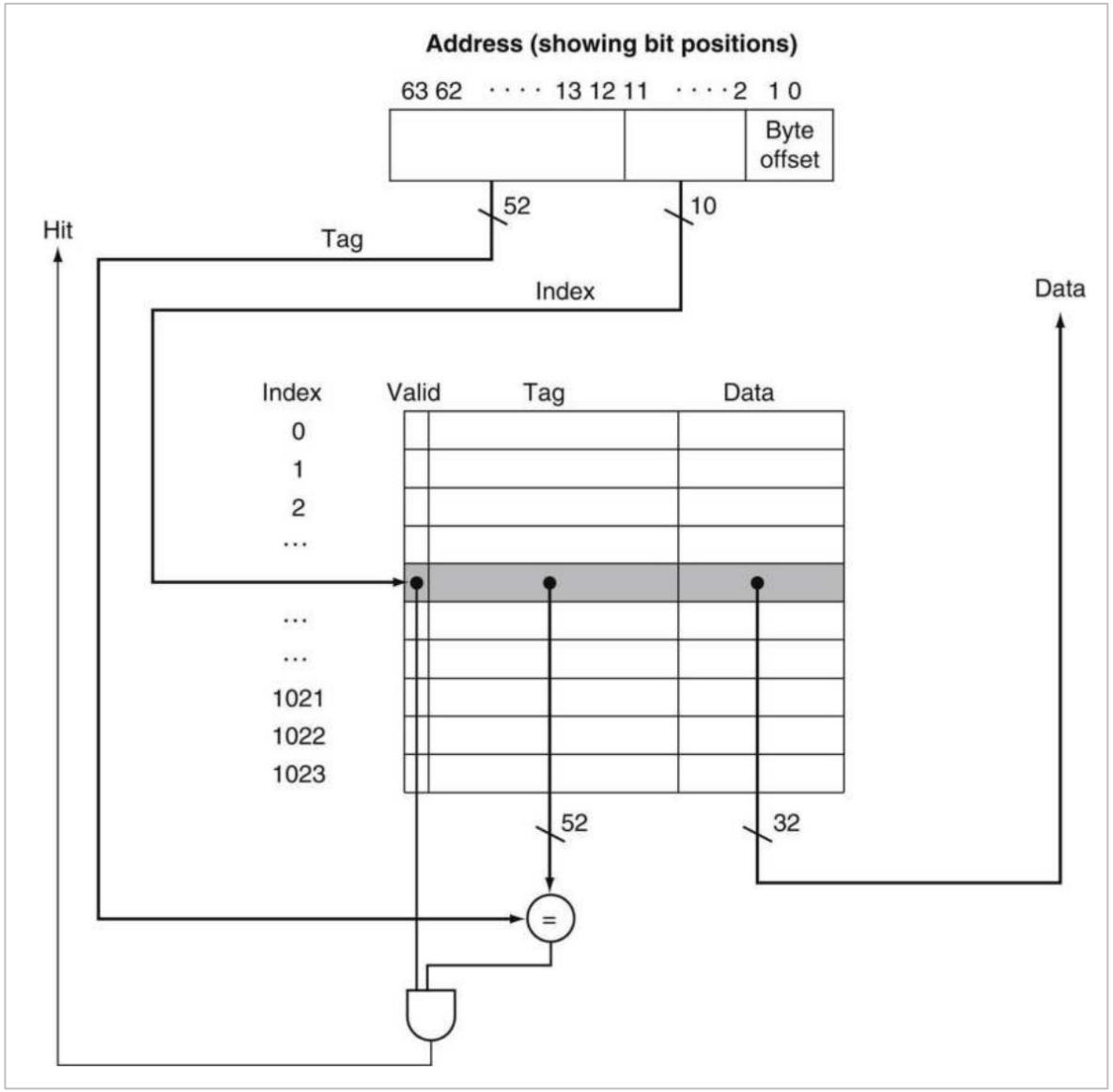
$$2^{10} \times (52 + 32 + 1)$$

Number of cache lines

bits needed for tag

bits needed for data
(4 bytes x 8 = 32)

bits for valid



Working of Direct Mapped Cache Memory

Decimal address of reference	Binary address of reference	Hit or miss in cache	Assigned cache block (where found or placed)
22	10110_{two}	miss (5.9b)	$(10110_{two} \bmod 8) = 110_{two}$
26	11010_{two}	miss (5.9c)	$(11010_{two} \bmod 8) = 010_{two}$
22	10110_{two}	hit	$(10110_{two} \bmod 8) = 110_{two}$
26	11010_{two}	hit	$(11010_{two} \bmod 8) = 010_{two}$
16	10000_{two}	miss (5.9d)	$(10000_{two} \bmod 8) = 000_{two}$
3	00011_{two}	miss (5.9e)	$(00011_{two} \bmod 8) = 011_{two}$
16	10000_{two}	hit	$(10000_{two} \bmod 8) = 000_{two}$
18	10010_{two}	miss (5.9f)	$(10010_{two} \bmod 8) = 010_{two}$
16	10000_{two}	hit	$(10000_{two} \bmod 8) = 000_{two}$

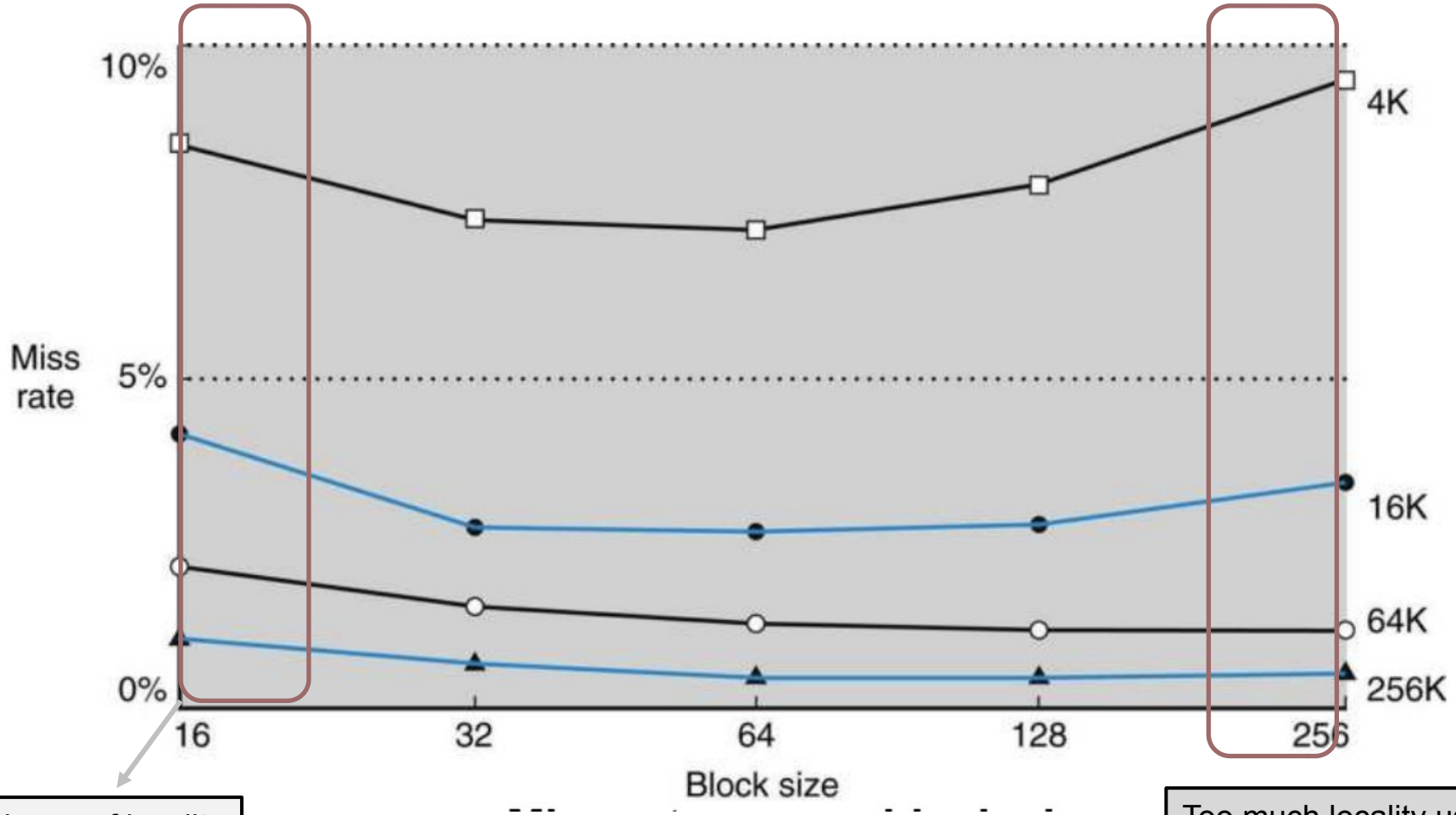
Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

a. The initial state of the cache after power-on

Index	V	Tag	Data
000	Y	10_{two}	Memory (10000_{two})
001	N		
010	Y	10_{two}	Memory (10010_{two})
011	Y	00_{two}	Memory (00011_{two})
100	N		
101	N		
110	Y	10_{two}	Memory (10110_{two})
111	N		

f. After handling a miss of address (10010_{two})

Cache Miss rate and Memory Block Size



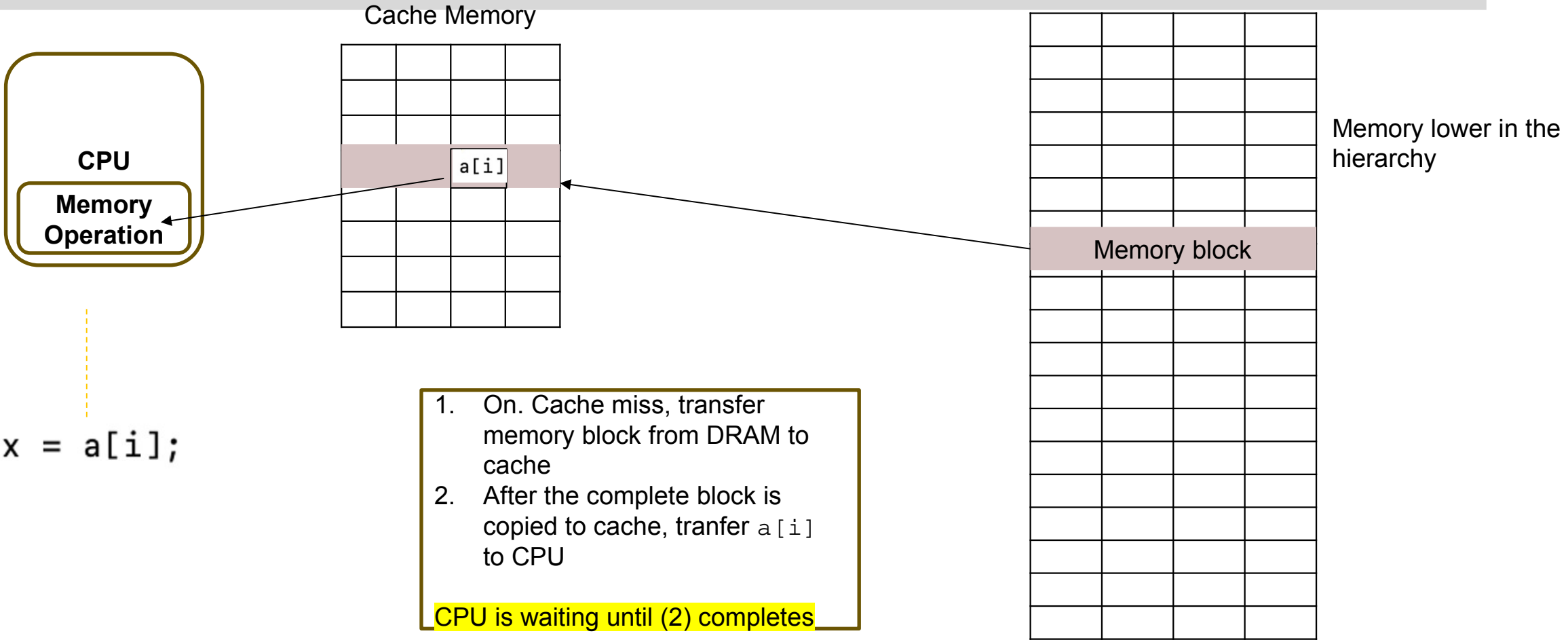
Not making enough use of locality

Too much locality used. Not enough lines in the cache → increased competition for cache lines →increased cache misses

Note that the miss rate actually goes up if the block size is too large relative to the cache size. Each line represents a cache of different size. (This figure is independent of associativity, discussed soon.)

Miss Penalty

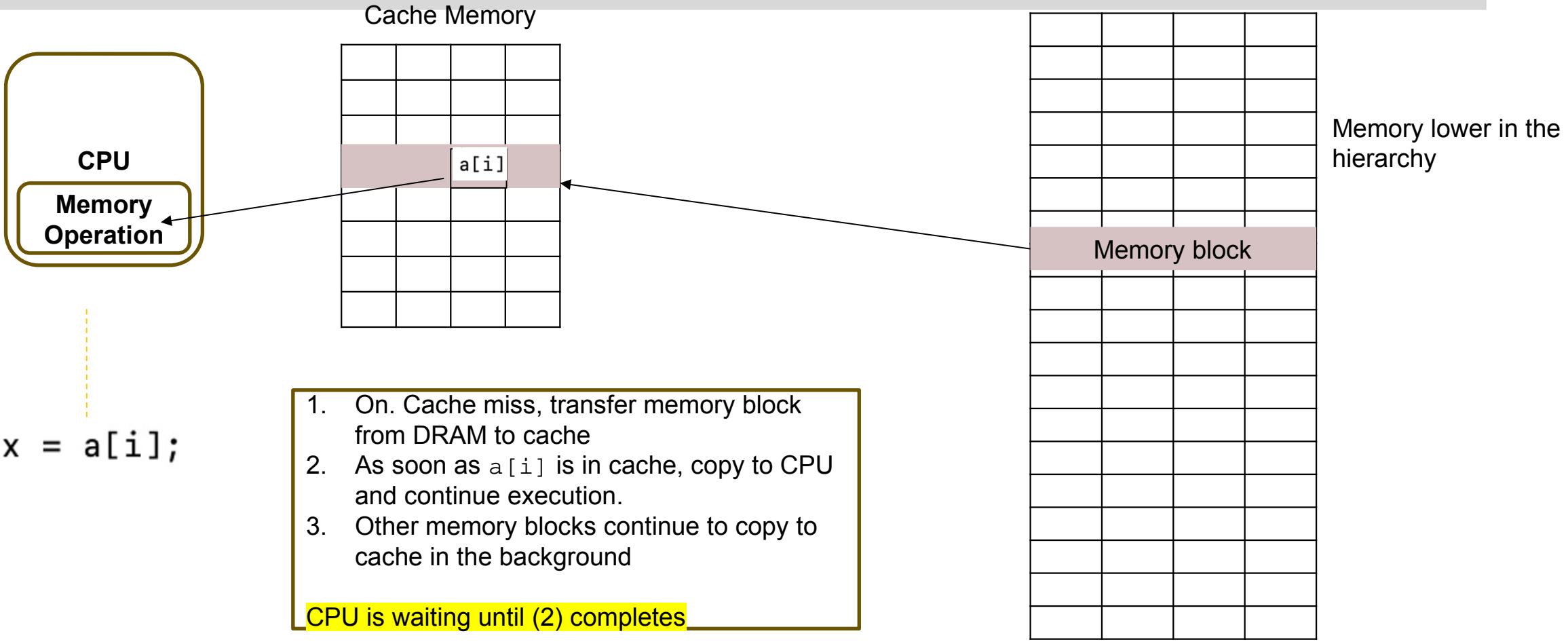
Miss Penalty: Time to fetch a block into the cache memory, when there is a cache miss.



On cache miss, larger block brought into cache → higher miss penalty

Reducing Miss Penalty: with early restart

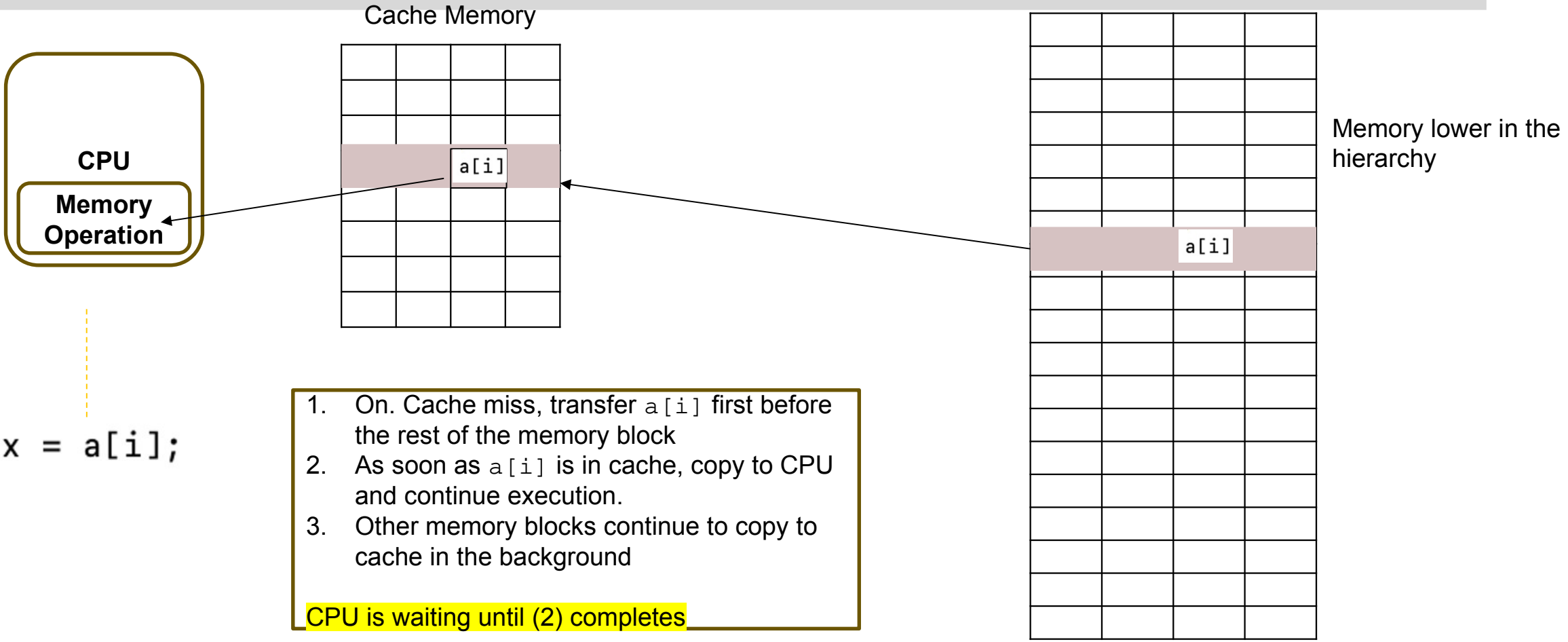
Miss Penalty: Time to fetch a block into the cache memory, when there is a cache miss.



Useful for caching instructions, which are mostly sequential

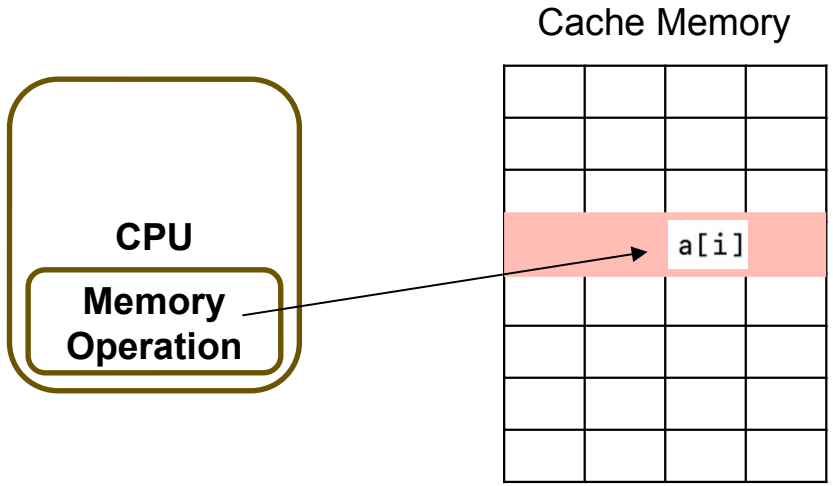
Reducing Miss Penalty: critical word first

Miss Penalty: Time to fetch a block into the cache memory, when there is a cache miss.

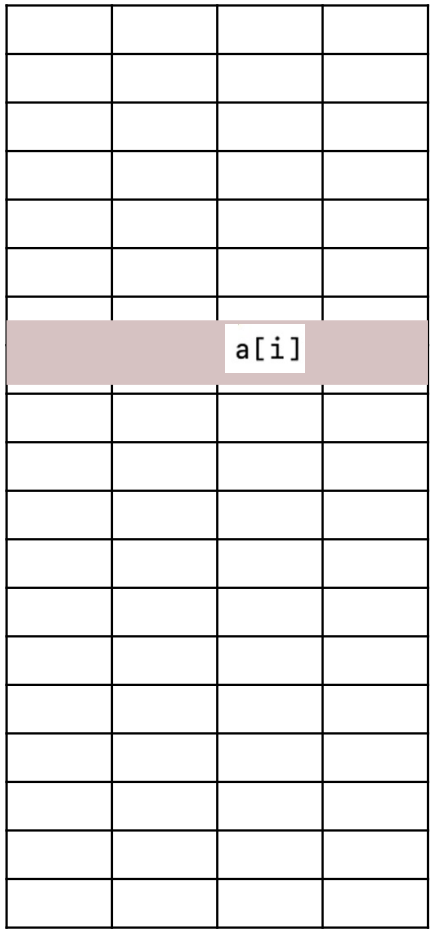


Useful for caching data, which have a more random access

Challenges with handling stores (writes)



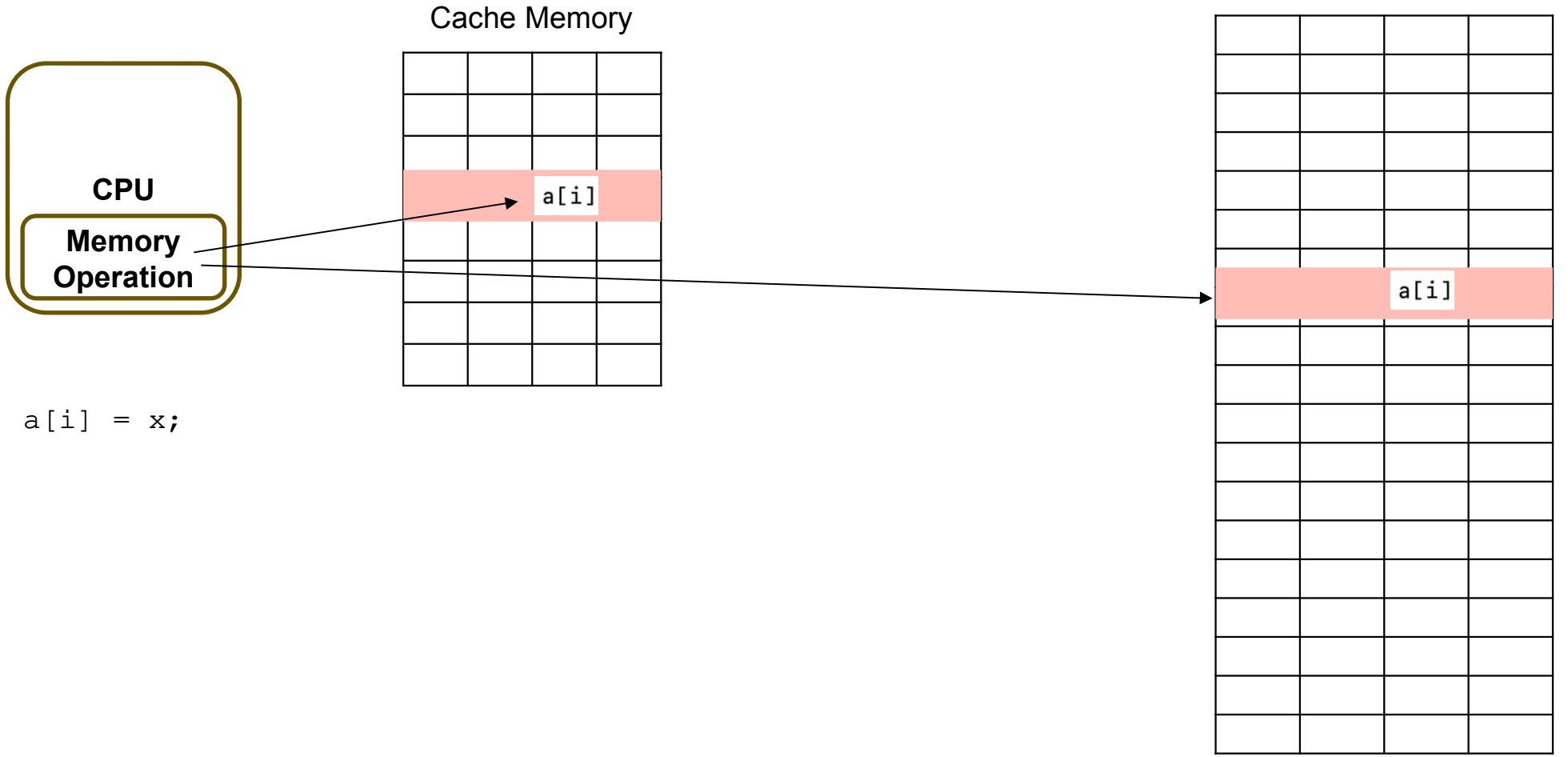

```
a[i] = x;
```



Memory lower in the hierarchy

Stores can cause incontinences: data in cache may be different from that of RAM

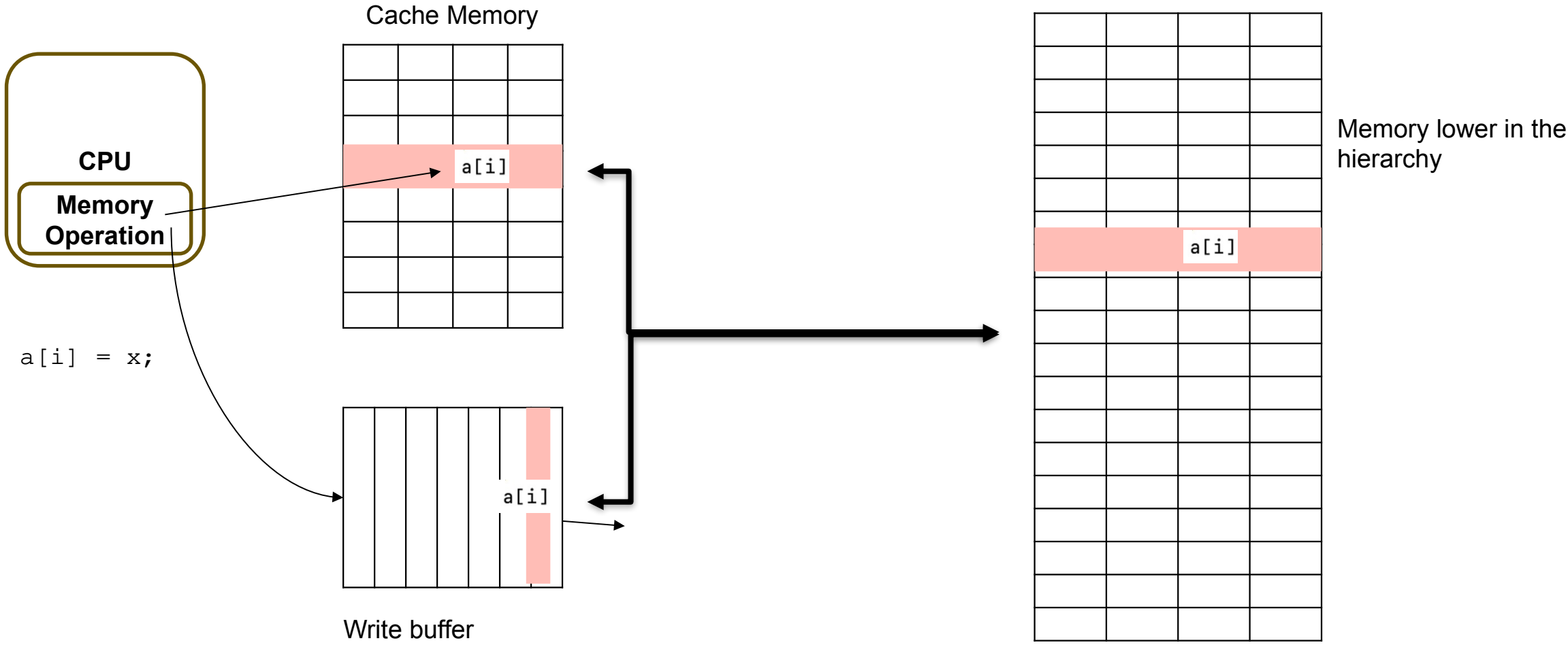
Handle writes with ‘write through’




```
a[i] = x;
```

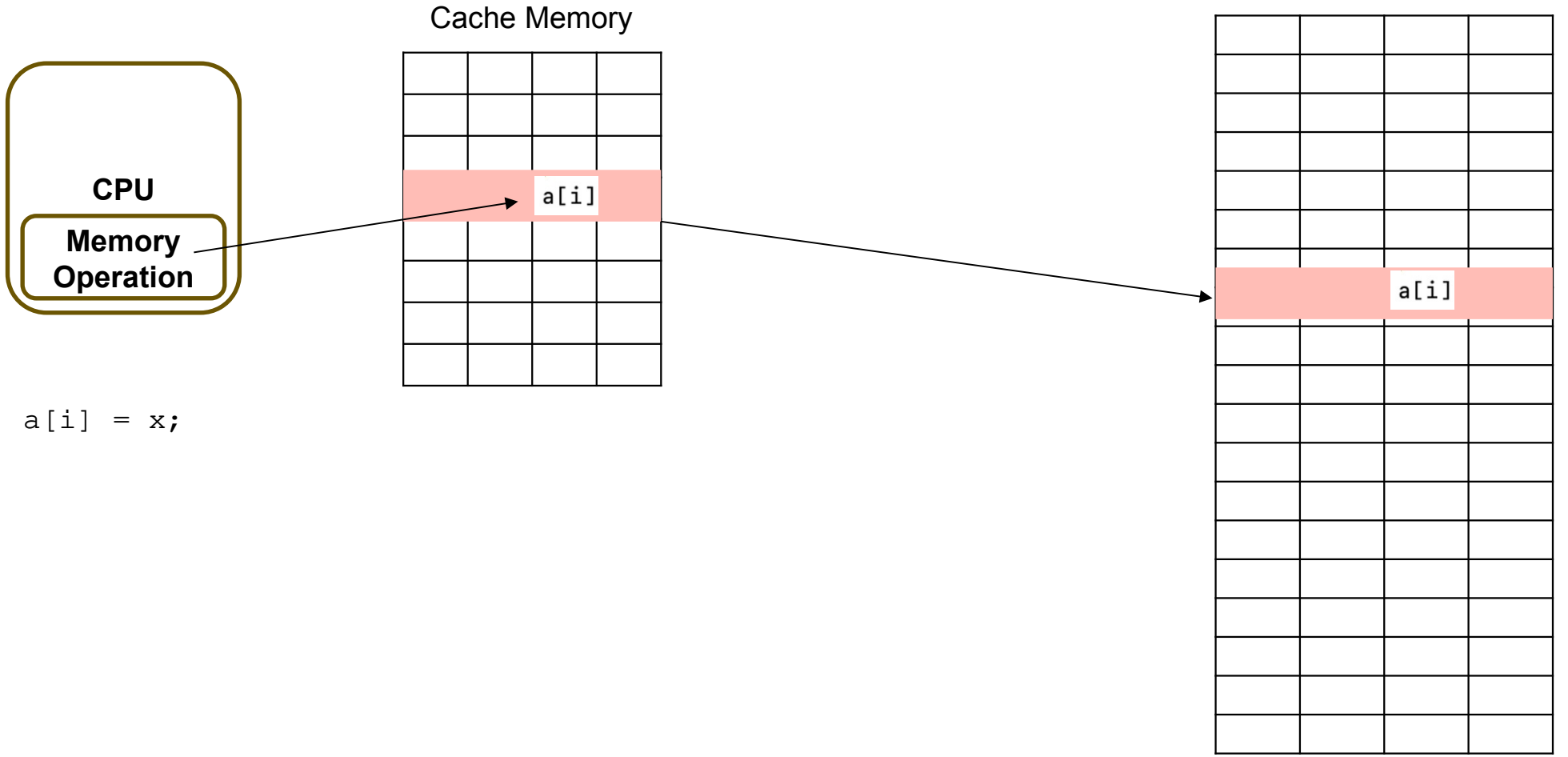
Any store updates all memories in the hierarchy. Pros: data is always consistent, Cons: CPU needs to wait for DRAM (very slow)

Improving ‘write through’ with a ‘write buffer’



CPU updates cache and write buffer, and continues its execution. Write buffer updates memory hierarchy. Size of write buffer must be large enough, to prevent CPU from waiting

Handle writes with ‘write back’

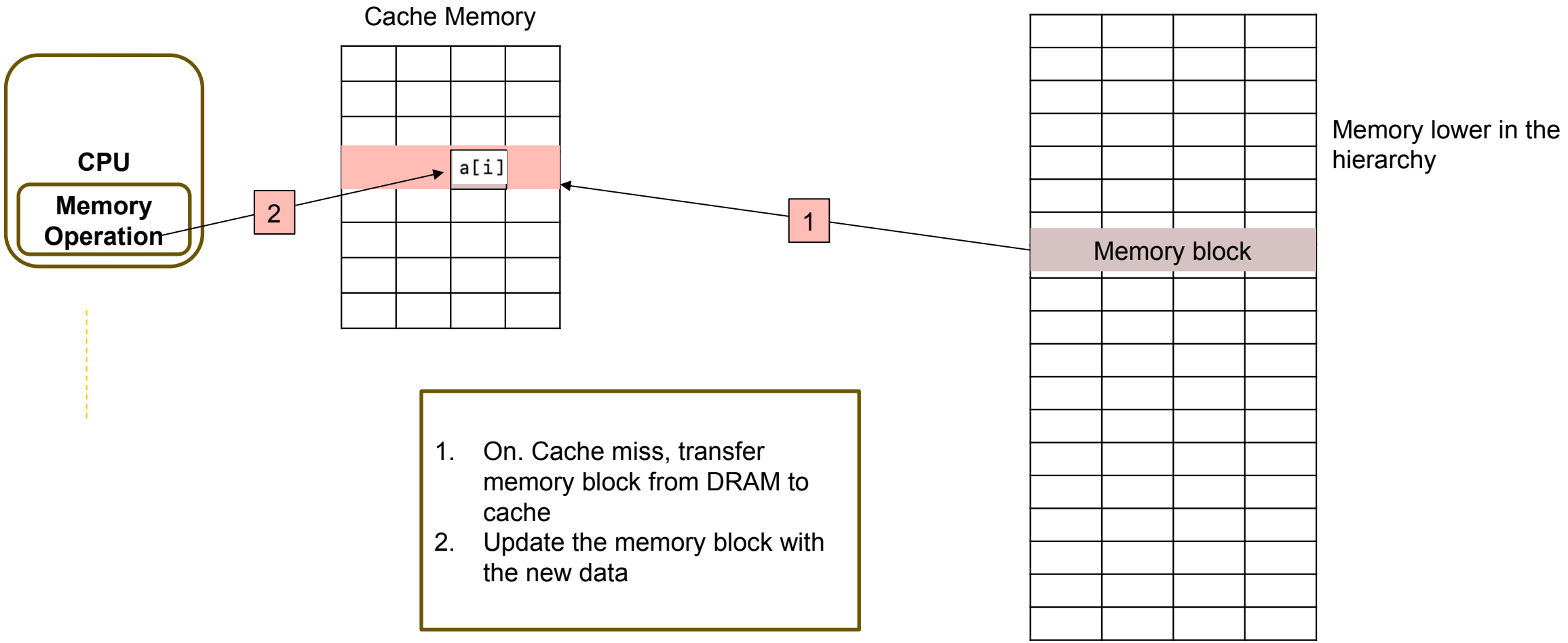



```
a[i] = x;
```

Modified data marked with a ‘D’irty bit. Written back to next level of memory only when it is replaced. More complex to implement

Write Misses, using write allocate

When a store instruction results in a cache miss



1. On. Cache miss, transfer memory block from DRAM to cache
2. Update the memory block with the new data

Cache Thrashing

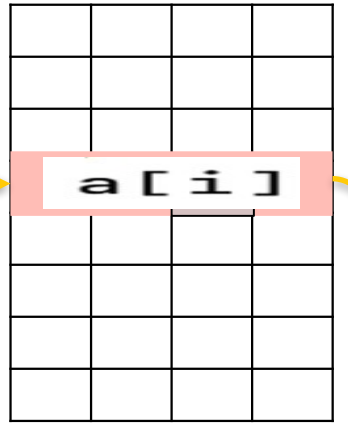
```
unsigned int  a[1000];

void traverse(){
   int i;
   int x = 0;

   for(i=0; i<1000; i+=8){
      x += a[i];
   }
}
```

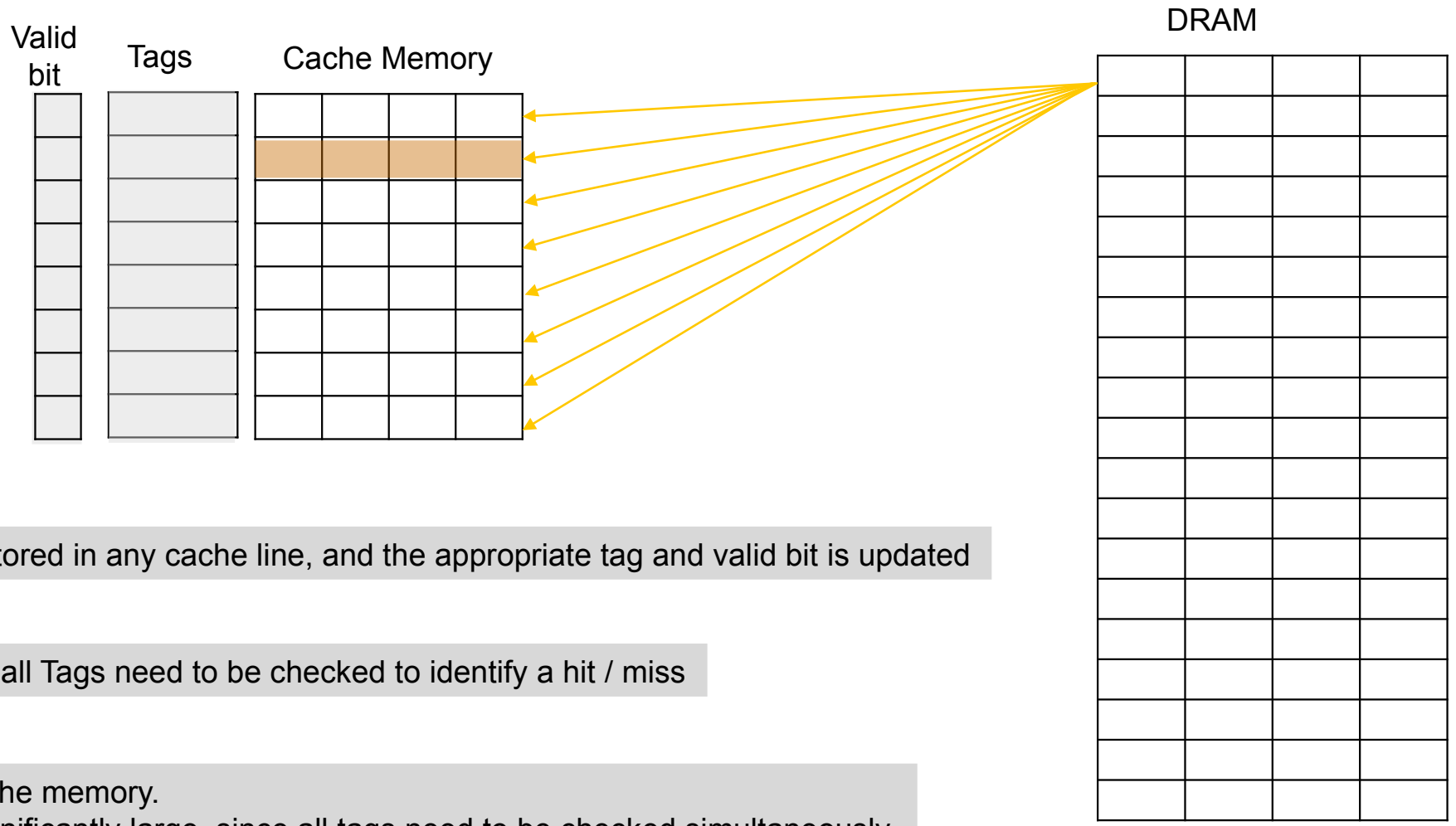
1 cache line = 4 words

Cache memory



The same cache line is used for all iterations! Each iteration of the for loop causes a cache miss. The other blocks in the cache are hardly used.

Reducing Cache Misses with Fully Associate Cache

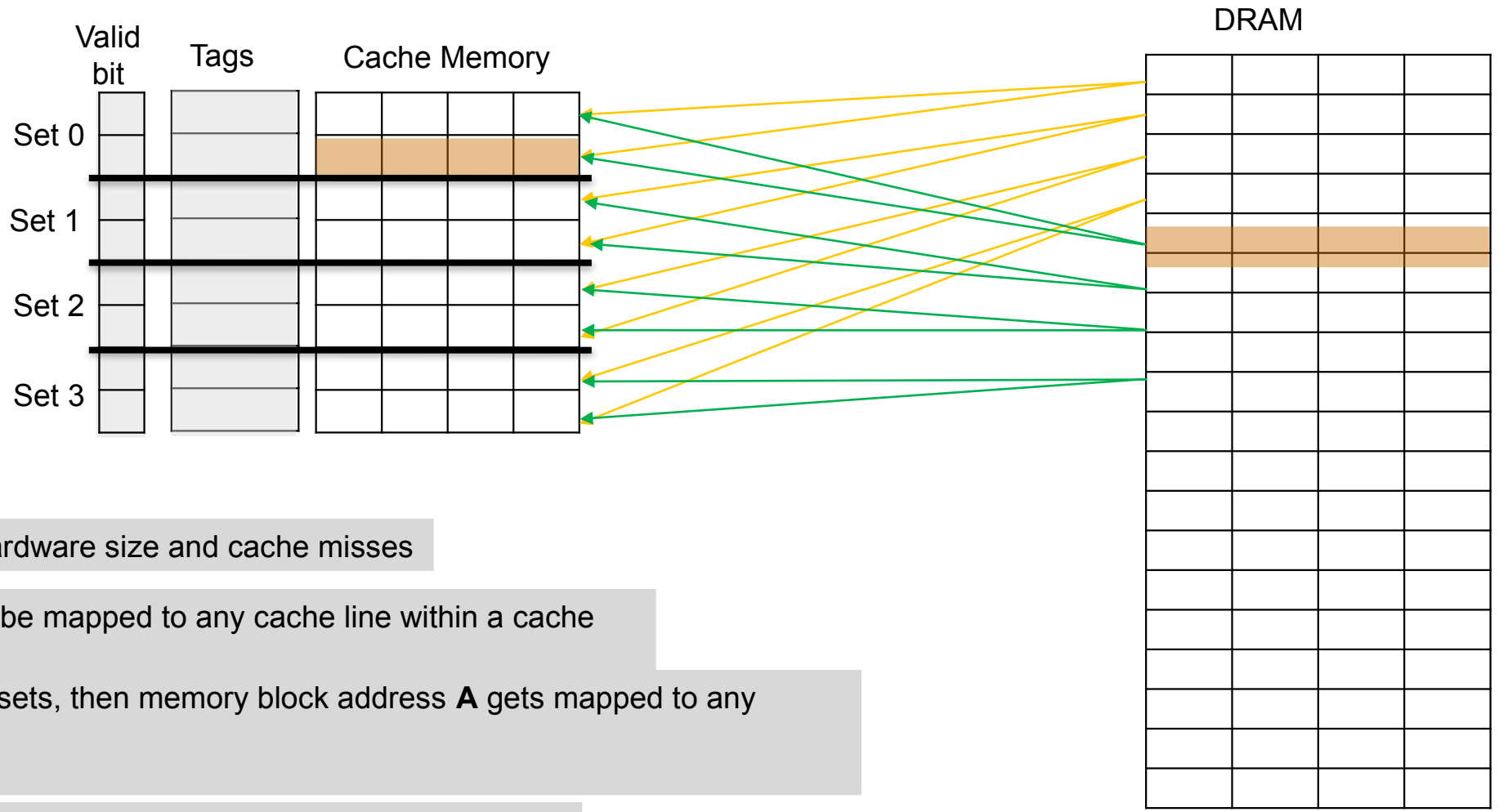


A memory block can be stored in any cache line, and the appropriate tag and valid bit is updated

If there is a read request, all Tags need to be checked to identify a hit / miss

Pros: Optimal use of cache memory.
Cons: Search logic is significantly large, since all tags need to be checked simultaneously

Set-Associative Cache



2-way
set associative

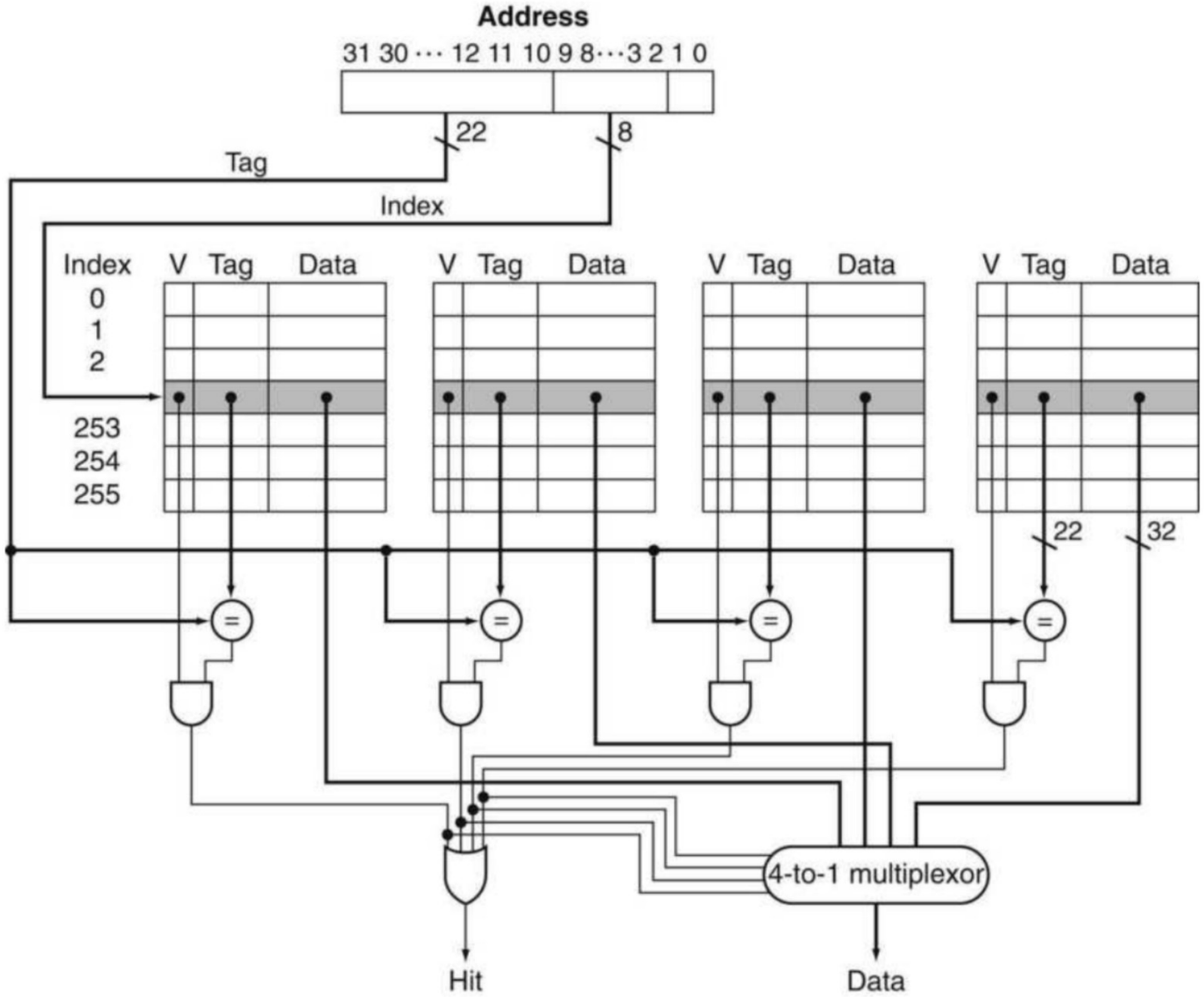
Trade off between hardware size and cache misses

A memory block can be mapped to any cache line within a cache set

If there are **S** cache sets, then memory block address **A** gets mapped to any cache
line in set **`A mod S.`**

Direct mapped is an example of a 1-way set associative cache
Fully associative is an example of an S-way set associative cache

A 4-way Set Associative Cache



Tag and Addresses

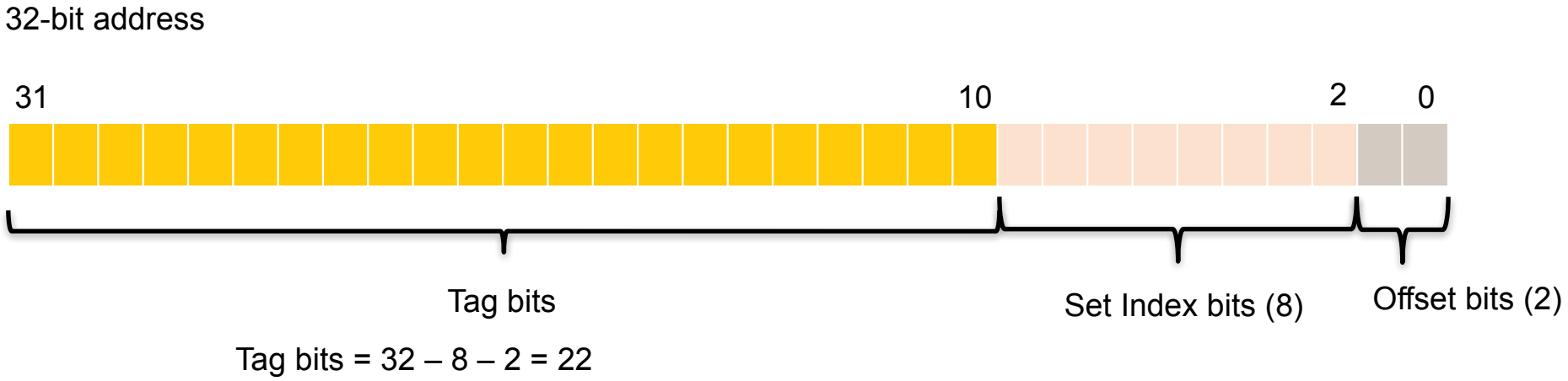
- Example: 4Kbyte 4-way set associative cache lines with each cache line holding 4-bytes

…. Number of cache lines = 4096/4 = 1024 cache lines

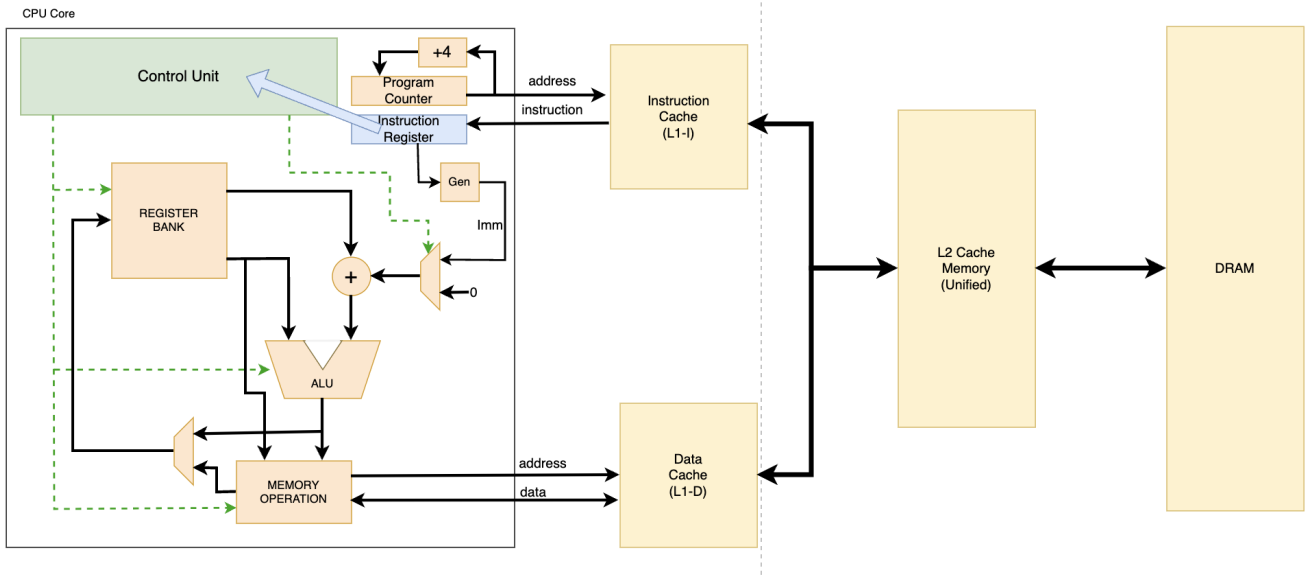
…. Each way has 1024/4 = 256 cache sets

…. $\log_2 256 = 8$ address bits required to identify the cache set

…. $\log_2 4 = 2$ address bits required to identify the offset within the cache line

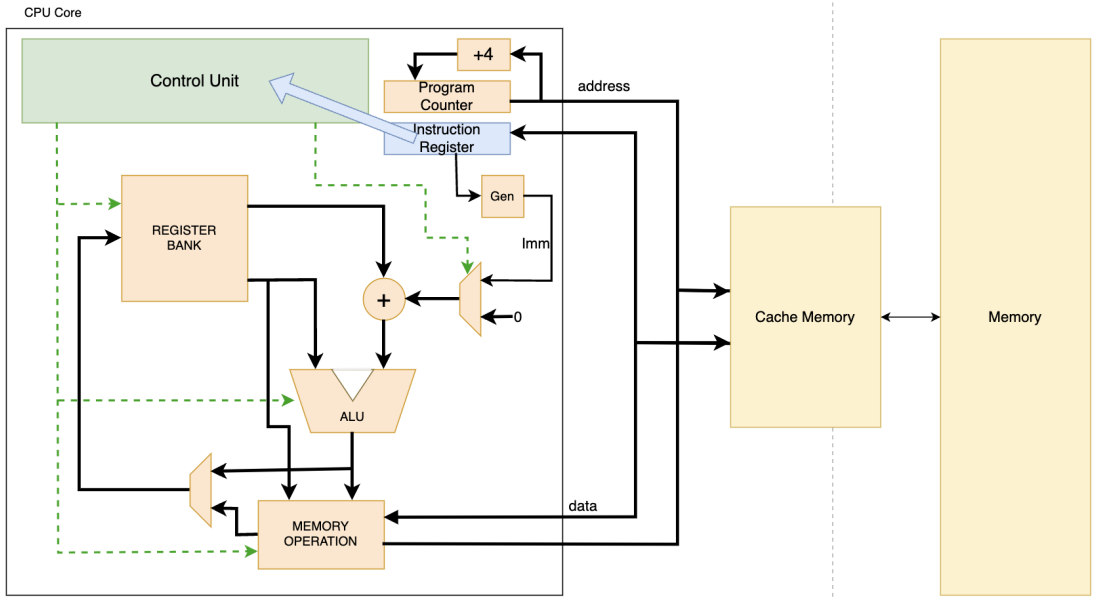


Split and Unified Caches



Split Cache

Better performance since an instruction and data can be loaded in same clock cycle (parallelization)



Unified Cache

Better utilization of cache resources

Split and Unified Caches

Split Cache

Instruction cache under used
while data cache is overused

Thus, several cache misses due to data cache

```
unsigned char a[1000];

void traverse(){
   int i;
   int x = 0;

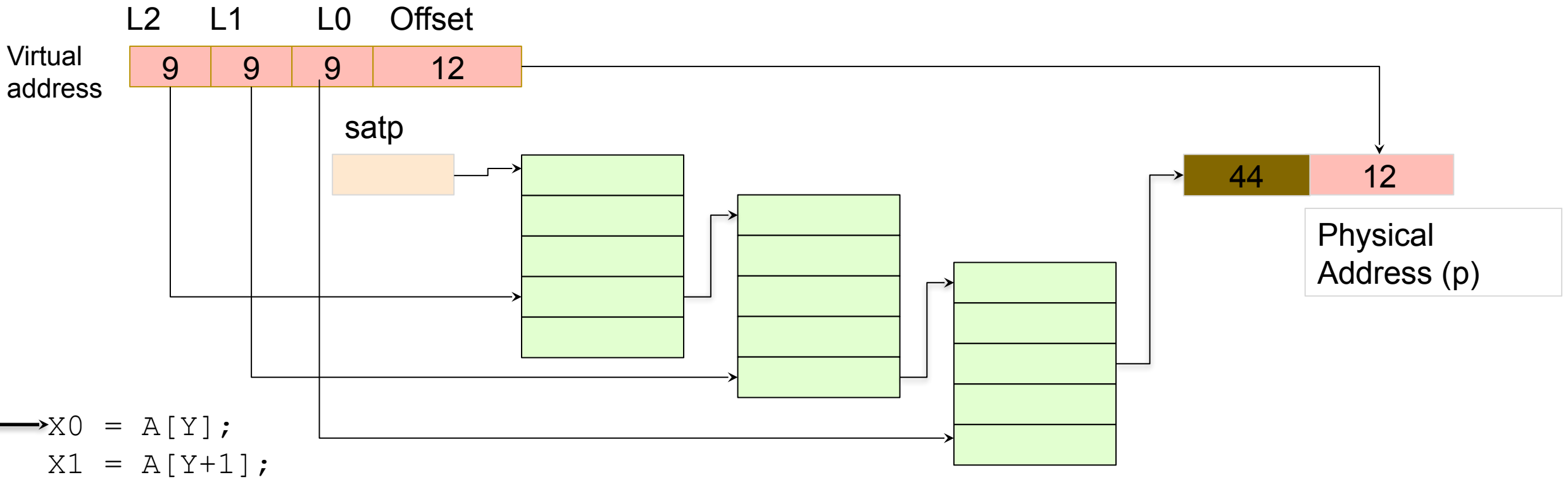
   for(i=0; i<1000; i+=8){
      x += a[i];
   }
}
```

Unified Cache

Instruction and data is same cache.
Optimal usage of cache.

Cache Memory and Paging

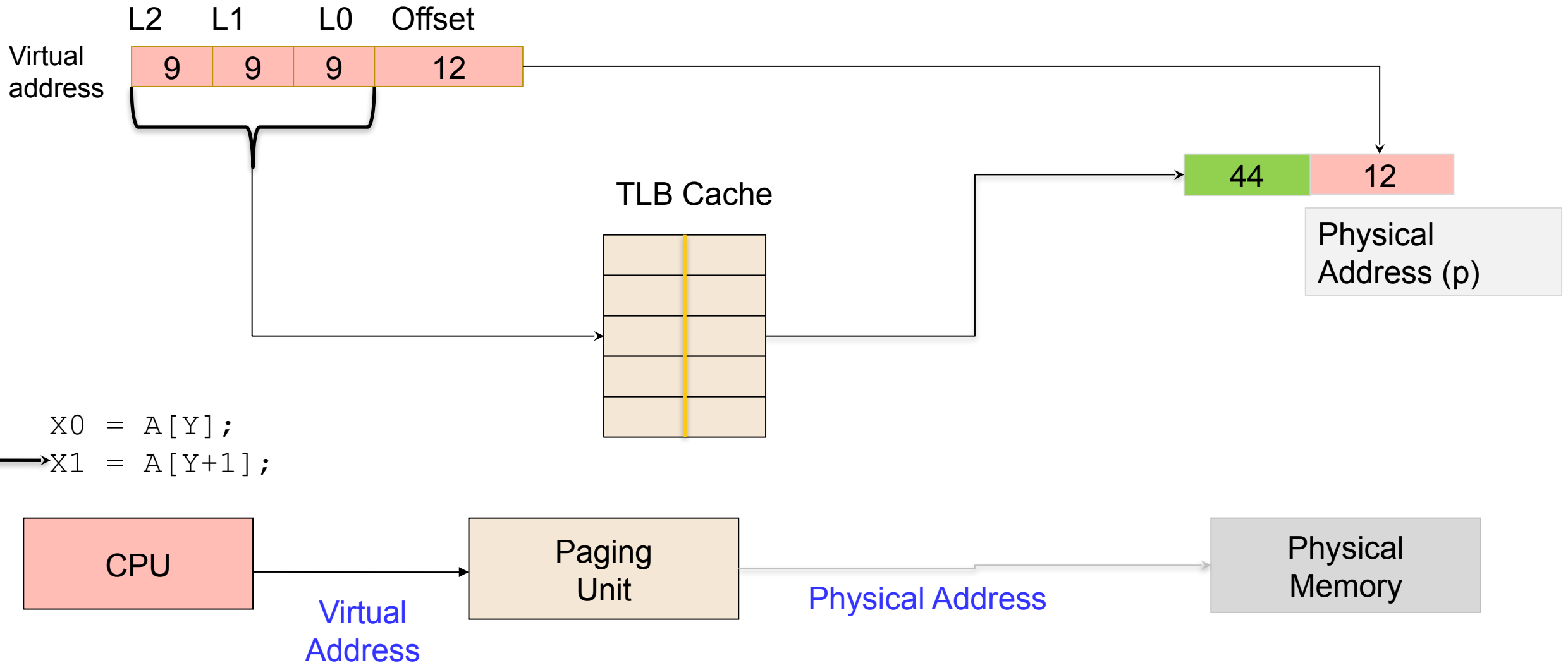
Virtual Memory Translation from a Virtual Address to Physical Address



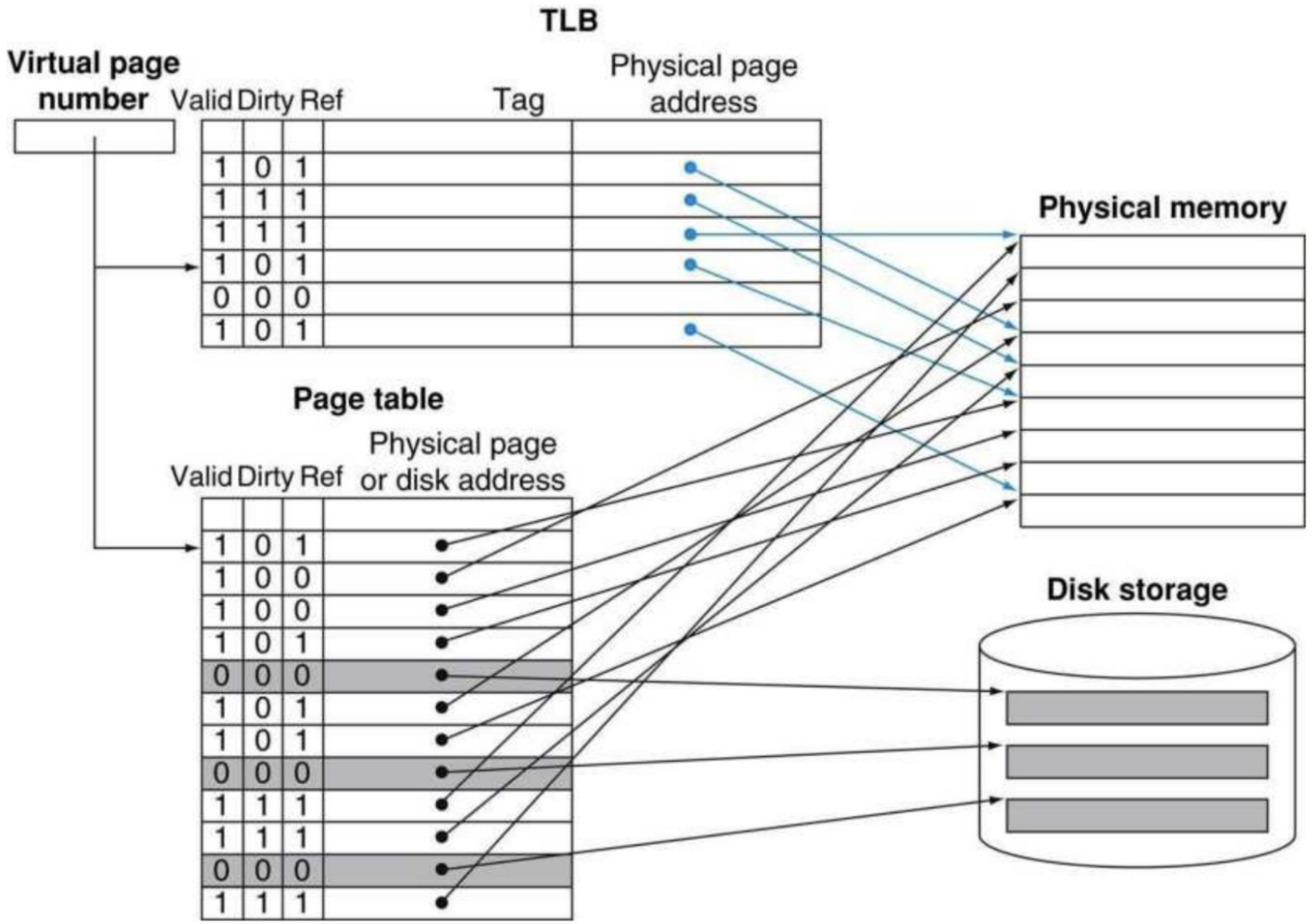


Recollect Page table translation in RISC V

Accelerating translation using a TLB

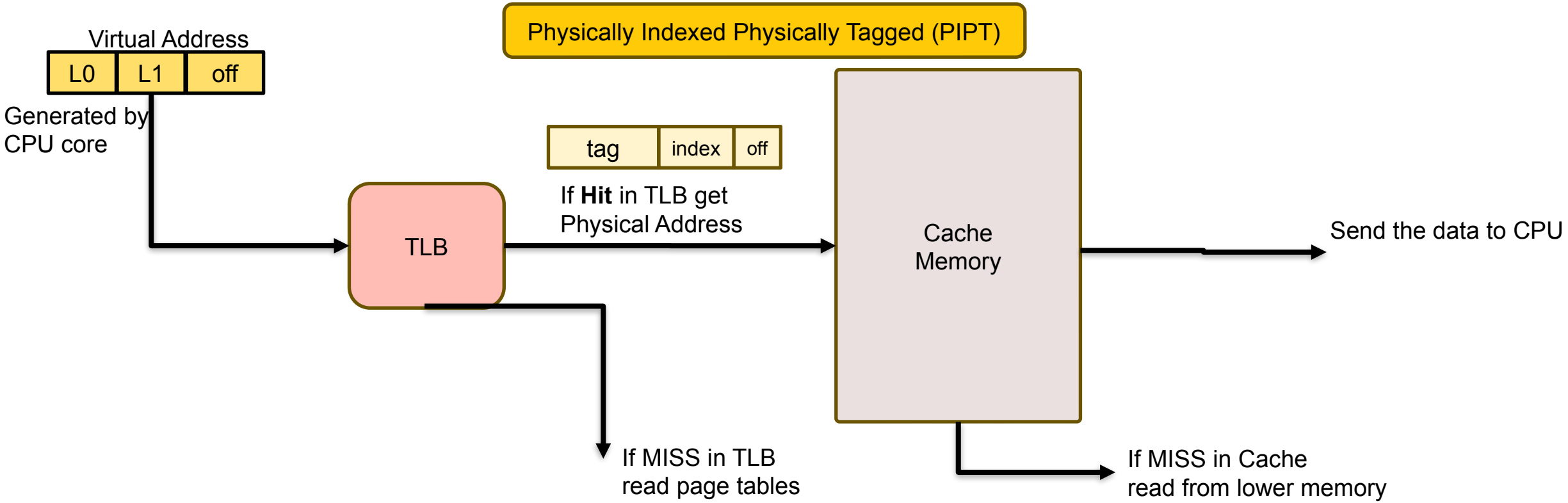


Closer look at Translation Lookaside Buffers



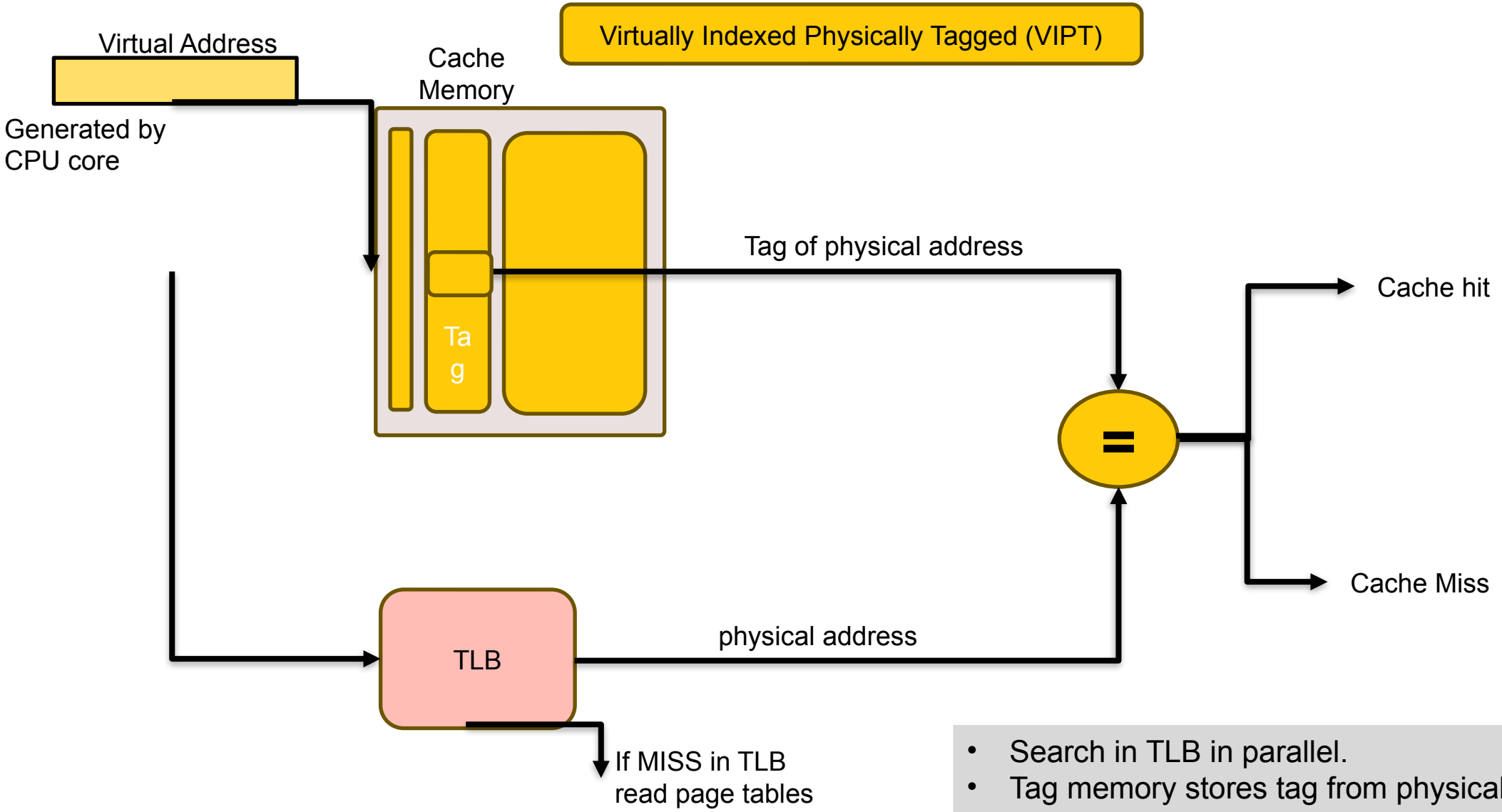
TLB is a small fully associate cache, which stores recently used physical page frame numbers.
Since it is a cache, it has a tag to identify the physical page address.
The tag is part of the virtual address.

TLBs and Cache Memories



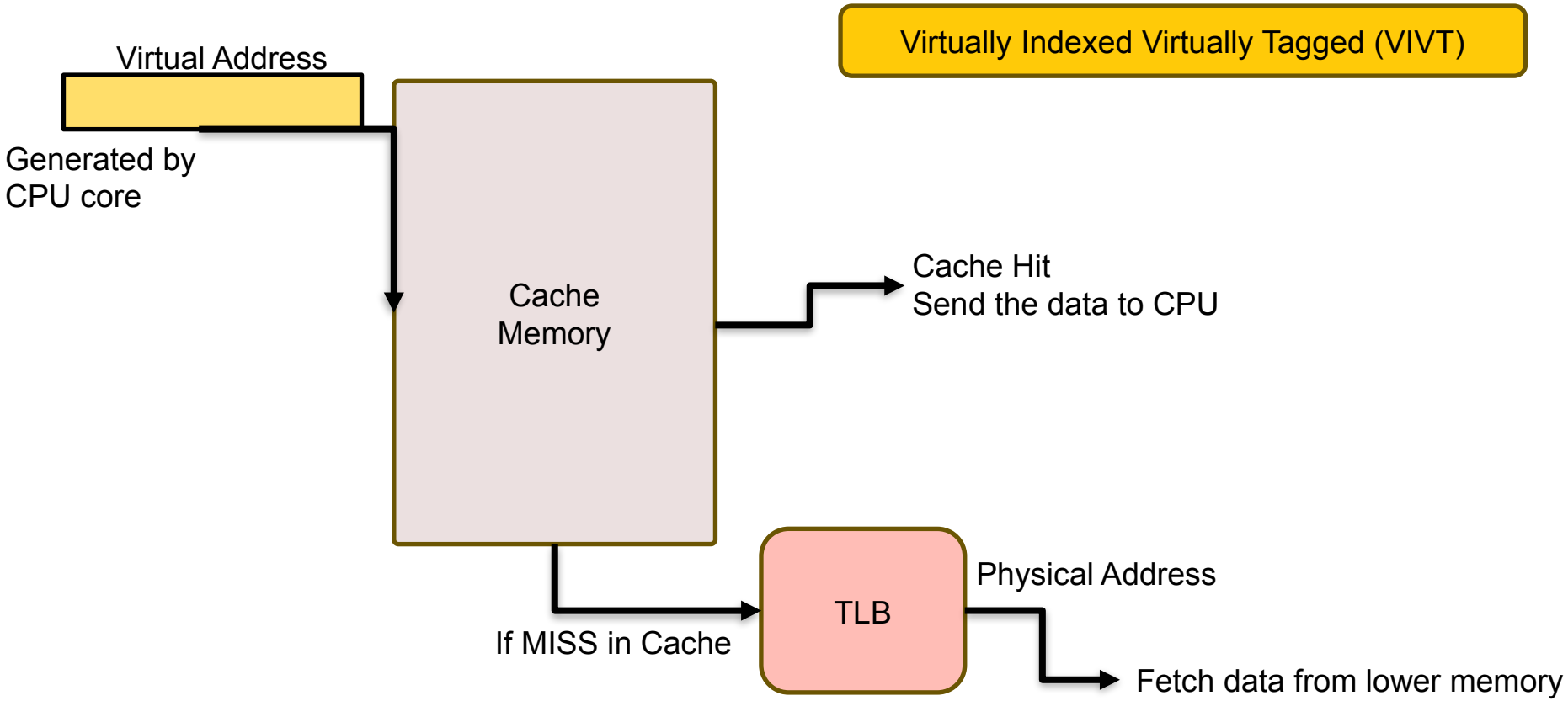
- Slow. Needs to wait for TLB and then cache read
- Useful for lower level cache memories (L3 for example)

TLBs and Cache Memories



- Search in TLB in parallel.
- Tag memory stores tag from physical address
- Useful for higher level cache memories (L1 for example)

TLBs and Cache Memories



Important information such as X,W,R bits that are stored in page tables are not accessible

Cache Performance

CPU Time = (CPU execution clock cycles + Memory-stall clock cycles) x Clock Cycle time

$$\text{Memory-stall clock cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

Fraction of instructions that are memory access

$$\text{Miss Rate} = \frac{\text{Misses}}{\text{Instruction}}$$

Example

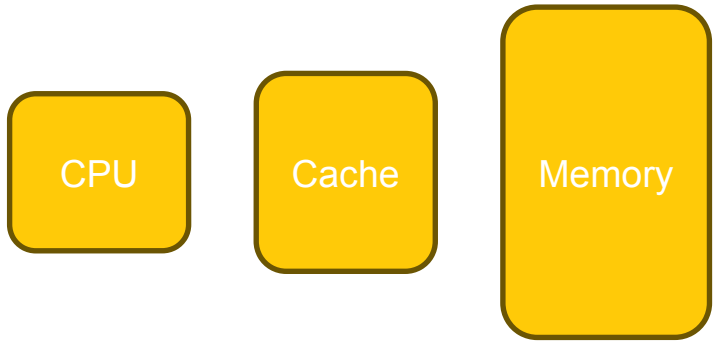
- A program has I instructions, of which 36% are memory operations.
- Miss rate of cache is 4%.
- Miss penalty is 150 cycles

Stalls due to memory operations = I x 36% x 4% x 150 = 1.56 x I

Example of Computing Cache Performance

- A program has I instructions, of which 36% are memory operations.
- Miss rate of cache is 4%.
- Miss penalty is 150 cycles

Stalls due to memory operations = (I x 36% x 4%) x 150 = 1.56 x I



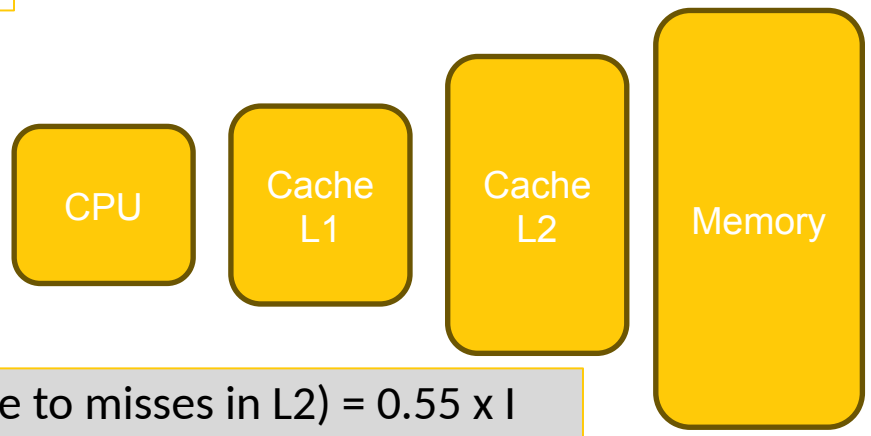
Usefulness of Memory Hierarchy

Example

- A program has I instructions, of which 36% are memory operations.
- Miss rate of L1 cache is 4%.
- Miss rate of L2 cache is 0.5%
- Reading from L2 is 20 clock cycles
- Reading from main memory is 150 cycles

Stalls due to Hits in the L2 cache is (4% - 0.5%) x 20 = 0.7

Stalls due to Misses in the L2 cache is (0.5%) x (20 + 150) = 0.85



Stalls due to Memory Accesses = I x 36% x (stalls due to hits in L2 + stalls due to misses in L2) = 0.55 x I